

# SKREM Alpha – A secure and practical SKREM-like symmetric key cipher, its premises and usages

Mircea-Adrian Digulescu

Current Affiliation: Independent

[mircea.digulescu@mail.ru](mailto:mircea.digulescu@mail.ru)

Previous Affiliation: Department of Computer Science, Faculty of Mathematics and Computer Science, University of Bucharest, Bucharest, Romania

[mircea.digulescu@my.fmi.unibuc.ro](mailto:mircea.digulescu@my.fmi.unibuc.ro)

February 2026

**Abstract** — In a prior paper we introduced a new symmetric key encryption scheme called Short Key Random Encryption Machine (SKREM), for which we claimed excellent security guarantees. With the advent of AI/ML based attacks, and leveraging HUMINT gathered in the interim, we now propose an improved version called SKREMAAlpha which is claimed to be secure against the new spectrum and also sufficiently practical. We discuss the decisions, assumptions, methods and data upon which the design and proposal are based. We also enumerate some use cases for SKREMAAlpha such as Keyed Probabilistically Random Function (KPRF), private secure cloud computing (FHE-like), changing private keys on a cipher without decryption and others. We discuss techniques and rough estimation methods for designing secure ciphers in the 21<sup>st</sup> century. Finally, we present some generic ideas for strengthening theoretical security further, including via Random Circuits and rewrite cryptography.

**Keywords**— *SKREM-like ciphers, Kolmogorov extractor, SKREMAAlpha, Random Circuit, SMT, AI/ML cryptographic attacks, symmetric key encryption, strong encryption, random expander Boolean circuits, rewrite cryptography, secure private cloud computation, cryptographic hash, censorship-free content distribution, Keyed Probabilistic Random Function.*

## I. INTRODUCTION

It's been close to 5 years since we introduced this new class of symmetric key ciphers called SKREM (Short Key Random Encryption Machine). The main novelty it introduced is pre-provided large truly random nonce, from which it gradually incorporated entropy to expand its internal secret pool and key size, up to the length of the plain text – thus functioning as a keyed Kolmogorov extractor over the nonce, generating new randomness. In a separate paper also circa 5 years old, we showed how such a SKREM cipher could be used in a host of cryptographic settings.

Since 2021, we the author were able to collect ample feedback, including HUMINT including from people associated with special services and western hemisphere agencies with competencies in encryption. The context of such frenemies encounters is a private matter and out of scope for this technical paper. However, based on those interactions, corroborated with individual theoretical research and given the advent of AI/ML cipher attacks, now, 5 years later we concede that the *particular version of*

*SKREM provided in the initial 2021 paper is NOT SECURE against state and superpower actors because of AI/ML distinguishing attacks.*

In this paper we present an improved and practical version called SKREMAAlpha of SKREM-like ciphers which is claimed to be secure even in the context of the new attack vectors. Additionally, it is claimed to be reasonably practical for many applications.

The novelties in brief introduced by SKREMAAlpha are:

- The Chaos Machine replaced with a CSPRNG based on modular inverse of a short dynamic polynomial over a large prime (8192 bits), reduction over a medium one (2048 bits), whitening using Keccak-f plus incremental, repeated, seed refresh using entropy gradually incorporated from the GMT nonce.
- Selection and Projection of bits from the GMT is improved.
- Two Random-G Turing Machine (RGTM) steps are introduced where random operations are performed by multiple threads on the same common memory which is initialized with intermediately produced random data. The RGTM is self-modifying and Turing-complete in instruction set.
- A Digulescian Permuter is used to enhance quality and do essentially bit mixing for intermediate outputs, over large bits sequences (16-256 MB).
- The short key (0.5-16KB) is expanded into an effective 16KB key based on which an effective original secret key of 32 KB is derived.
- SKREMAAlpha operates on blocks of 2 GB (corresponding each to 16 MB of encrypted input). Each block produces a 16MB secret key seed to be used to get the next blocks' effective key.
- The first block does not encrypt plain text, but is used as a Key Derivation Block, generating a 32 MB original key secret pool plus a 16 MB secret key seed for next (SKS) block.
- Plain-text encrypting blocks each have an incoming effective key which is plain-text sized (16MB) which is derived based on the 16MB SKS, the original 32 KB expanded key and the original key secret pool 32 MB.

We also discuss decisions, assumptions and claims for SKREMAAlpha.

Later on, we also discuss a scheme for evaluating security of a cipher in the new attack environment post AI/ML based solvers.

We briefly discuss conclusions of research and HUMINT activity which lead to the evolution of SKREMAAlpha.

Finally, we speculate on the security of mainstream current ciphers and project the evolution of cryptography for the rest of the 21<sup>st</sup> century.

The paper is concluded with a brief overview of avenues for strengthening SKREMAAlpha, including repeated applications to prefix and suffix its data produce' via expander random Boolean circuits and posing the question if making use of rewrite cryptography can ever make sense.

#### A. Prior work

We the Author introduced SKREM-like ciphers in 2021 in [1] and also presented detailed use case scenarios in [2]. The core idea was to use a large-sized public nonce from which to import entropy, in order to greatly expand a secret internal state. While many ciphers including AES [3] support the idea of a short nonce, usually key-sized to 128-256 bits, even today, 5 years on, to the best of our knowledge no such system has been proposed before or since. Nevertheless, we, the author, strongly suspect that non-public research, by researchers such as Marius Zimand (see [4] and [5]) or Leonid Levin (see [6]) might include something similar to SKREM. The original variant of SKREM introduced in [1] made use of the Chaos Machine introduced by Armour in [7] as a randomness expander. While a NIST standard, the Chaos Machine approached proved, based on credible digest of HUMINT data and more, to be insecure. The original SKREM also suffered from a number of beginner omissions, mainly lack of bit mixing and low diffusion / confusion in the actual straight-forward manner proposed for importing entropy from the nonce.

The public domain publications over the past 5 years have, to the very minimum, confirmed to our satisfaction, that in effect all “short, simple and fast” ciphers, which includes with the exception of McEliece [8] basically all currently used cryptographic primitives, like AES [3], Serpent, the newer Keccak-f primitives [9] and basically all, are vulnerable to a new type of attack based leveraging advances in AI/ML. Systems like SMT Solvers [10] [11] and others can be used to deduce “correlations” between parts of input and parts of output so as to enable distinguishing and similar keys attack. Before the advent of effective AI/ML tools, it was practically humanly unfeasible for a researcher to explore the problem domain of such by hand.

Some successful attacks based on the above include those on reduced versions of AES [12] [13] [16], on SPECK [14] [15] and others. It is very likely in the author's judgment, especially corroborated with HUMINT and OSINT he gathered, that to a datacenter sized adversary, with ample precomputation time measured in years and decades, just like with AI/ML, so with attacking (full round versions) of mainstream ciphers like AES – doable and doable fast (20-60 minutes).

Recent results have shown successful subexponential distinguishing attacks even on McEliece [17].

Based on all the processed data we the author have gathered and processed, we conclude that no “short, simple and fast” cipher can be expected to be secure. So what can be secure in the 21st century are ciphers with a very large block size MB-GB, are based on algorithms which are long and not simple, namely they incorporate randomness from a large public nonces which are MB to GB sized, involve non-classical, non-algebraic transformations including those based on using priorly grown secret randomness (getting as close as possible to a truly Random Expander Boolean circuit [18] [19] [20] and qualitatively improve over short-seed CSPRNGs even those proposed in [21]), and finally run in a non-trivial amount of time, measured in seconds and minutes or even hours or days, not in milliseconds.

If any short, non-uniform algorithm which leverages randomness only as input, can be devised by a human researcher, so that it is secure, we claim that then SKREMAAlpha is secure, and that furthermore its security will last throughout the turn of the current 21st century.

#### B. Overview of this paper

The rest of the paper is organized as follows.

In Section 2 we recap the SKREM cipher in a nutshell.

In Section 3 we present SKREMAAlpha and its constituent components.

In Section 4 we discuss aspects and decisions pertaining to SKREMAAlpha. We also draw some preliminary conclusions and formulate security claims. Additionally we enunciate and describe in sufficient details some use cases enabled by SKREMAAlpha, including private secure FHE-like cloud computing.

In Section 5, we comment on OSINT, HUMINT and other sources of inspiration for the ideas in this paper. Additionally we provide claims, conjectures and educated predictions about the evolution of cryptography and ciphers over the rest of the current 21st century.

In Section 6 we present some minimalistic yet comprehensive and typical initial quality stress testing proposals on SKREMAAlpha and its components.

In Section 7, we introduce a formula to be used in the future for ascertainment of relative security of any encryption or security scheme based on several factors. It is indirectly inspired by Carl Sagan's Fermi Paradox resolution formula for estimation of number of existent civilizations.

In Section 8, we present a series of enhancements which can be used in conjunction with SKREMAAlpha (or any similar KPRF): expander random Boolean circuits and rewrite based cryptography. We also present a general template for incorporating generic enhancements.

Section 9 offers a brief Vitae of the author.

We conclude the paper with Acknowledgments in Section 10.

## II. RECAP OF SKREM-LIKE SYMMETRIC KEY ENCRYPTION CIPHERS

We introduced SKREM-like ciphers publicly in 2021 [1], based on ideas developed circa 2015. Its main novelty was the use of a truly-random large public nonce (think 1 GB), a

short encryption key and a relatively simple algorithm (new-randomness Kolmogorov extractor), by which entropy from the public nonce was gradually imported into a large secret state within the SKREM-like cipher. Once that state got large enough, it was XORed with the plain text (actually the nonce modified to give out the state of the proper plain text). The novelty was this use of a truly random large public nonce (unique and different for each encryption), and the concept of importing entropy from it based on a private key – thus obtaining a KPRF (Keyed Probabilistic Random Function).

Additional novelty was in the use of the INDIRECT operation: using some secret state of 32 bits to “indirect” (address) two locations into the public nonce (or any truly random source, regardless of bit distribution) and use the result of the comparison of the values thereat stored (2-8 bits in size) to get a new uniformly distributed secret bit. The INDIRECT operation was meant to introduce highly non-linear, non-algebraic correlations.

The above ideas and concept continue to be novel to mainstream public (open access) cryptography, and are sound and valuable in themselves.

Unfortunately, the original variant of SKREM (which was intended to be just a Proof of Concept of existence of SKREM-like ciphers) suffered from a host of “beginner mistakes” the most important being lack of bit mixing. These omissions, coupled with advances in automatic AI/ML-based SAT Solvers (such as those used to reliably attack SPECK [14] and reduced variants of AES [12]) make the original variant INSCURE. This assessment is based also on HUMINT gathered from statements from secret services of various nations, and from “the 2022 Lotte event” based on exfiltration of Western Block (USA/UK/other) persons with direct involvement in cryptography and related activities for top-level organizations of western power.

Now, 5 years on from the original release of SKREM-like ciphers, we are ready to introduce a claimed secure variant called SKREMAAlpha, based on the lessons learned (theoretical reflection), analysis of OSINT data pertaining to changes in attack vectors, especially advent of AI/ML systems for offensive cryptography, and exfiltrated HUMINT data pertaining to how such offensive systems are actually contrived and what can be done to make ciphers secure against them.

In Section 5 we will discuss some technical findings pertaining to symmetric key cryptography learned however, in the context of claims, conjectures, and predictions.

The main lesson learned from the timeframe 2021-2026, pertaining to symmetric key cipher is that “almost nothing is secure under modern AI/ML distinguishing attacks from an adversary with vast compute power and ample preprocessing time”.

### III. THE 20 STAGES OF SKREMAAlpha V1.0

This section describes the core of SKREMAAlpha.

Similarly to the story of Bitcoin founder, the author had to start implementing SKREMAAlpha in C# to convince himself that the following general approach was both specifiable, implementable and sufficiently performant to fit

into reasonable (if but generous) memory and running time limits. At the time of writing this paper, all the steps have been specified in a C# repository in a top-down approach, except for the Random-G machine details, the key expansion steps, the filling in of the final results and the modification of the incoming nonce for encryption purposes (all of these being considered sufficiently clearly specified not to require actual implementation to be published).

Specific details of the components used such as the ComplexStateCryptographicallySecureRandomNumberGenerator (CSCSPRNG) which is described below, fully are not present in the current version of the implementation. Lower-level details pertaining to RSA/good prime finding, Big Number operations, Buffer Operations, RandomPermuter actual implementation are also omitted (their realization being still obvious in incarnation from the descriptions below). Nevertheless what was implemented suffices to convince the author of the reality and implementability of SKREMAAlpha.

The author in fact, is actively working to complete SKREMAAlpha implementation.. however, like with most IT projects, doing the scientific core part is just 20-30% of the whole work quantity-wise.. and here we can say that a full, working implementation of SKREMAAlpha, done by a single software engineer, working alone, could easily take 2-4 quarters of full time work (including some basic testing). The author is seeking financing for this project in context of S-Company [22] and M-Company [23].

The current state of the repository is available on the author’s GitHub profile as a project [24]. Please stand advised that the implementation in the repository is not complete, the numbering of the stages differs slightly from the below and their effective contents is also sometimes slightly different from what was specified here. The implementation sketch in the repository preceded this paper.

Building on SKREM-like ciphers, SKREMAAlpha takes the following input and produces a single large cryptotext as output:

- A secret key of short, but not necessarily trivial size (so 0.5-16 KB in the current variant)
- A large (circa 2 GB), truly random uniformly sampled *secret nonce* called grand master table (GMT) unique and different for each encryption, which except for a few minor bit changes (about plain-text length), will be published as is, as *the cryptotext*.

SKREMAAlpha works as follows:

0. The short secret key, if it is less than 16KB in length is expanded via some or a combination of strategies (used in sequence): splitting the key into 256 bit sequences and appending the bits 100-356 from the square root of the first nearest prime to the key pool, inverting module some prime deduced from half the key size, using a sponge function, and **importing some fixed data from the GMT nonce to 2x-4x the key size**. Then, a “round key” consisting of at least 32KB is derived from it by

some classical method (eg. Keccak-f sponge construction) and is split in parts used to seed and initialize the first ComplexStateCSPRNG (CSCSPRNG).

1. It gradually imports entropy from the public (ultimately slightly modified, but wrt to initial steps identical) public nonce, using a new algorithm, different from initial SKREM variant. This algorithm is based on something called a ComplexStateCSPRNG (CSCSPRNG) and proceeds as follows:
  - a. A value is sampled from the CSCSPRNG which is actually something like ~32k bits long.
  - b. These bits are used to *select* a portion of the nonce splitting the respective sequence into two parts: one corresponding to the 1s in the relative position in the CSCSPRNG bits, and the other to the 0s.
  - c. The selected part with the 0s is left unmodified (*untouched*) and it is simply moved around or considered only in later stages. The other part gives out ~16k bits of nonce-based randomness. The indexes (bit locations) for both parts are never reused again in the algorithm basically (are removed from consideration).
  - d. The selected bits are first whitened using the CSCSPRNG's Fast sponge function and then *projected* over the outstanding untouched portion of the nonce – meaning they are used, in conjunction with a shorter CSCSPRNG to reference some indexes in the GMT and, based on the comparison of the bit values there, to generate ~120 *strong bits* (which are also uniformly distributed).
  - e. Part of these  $10240 + 120$  bits are fed back into the CSCSPRNG as fresh refresh randomness under a specific strategy (Subsection B). The other part is used to “expand” a secret internal state for the stage.
  - f. Note: The CSCSPRNG is seeded based the secret key initially, and partly reseeded as per point e) above with new randomness derived from the GMT (based on GMT and the secret key).
2. Once sufficient bits are gathered in the internal secret state, the first 1MB of them are discarded and a “Digulescian Permutation” is applied to (1) the internal state and (2) the untouched bits. This shuffle is basically a cycle-notation based Schiffer shuffle which is derived using a new fixed-state CSCSPRNG called a RandomPermuter. This RandomPermuter takes 1-5 CSCSPRNGs as arguments (essentially producing a mix of 3-5 shuffles), ranging each in high entropy bits from 0.5-16KB for the small permutation (the initial secret key size – bits are not used directly, but are extracted from the private key alone), to circa 128-256MB of extracted *selected affirmative* (as opposed to untouched) bits. So the imported randomness from the GMT nonce is used to permute both itself and the untouched portion, in conjunction with bits of the private key.
3. The permuted *selected affirmative* bits are “enhanced” by being *projected over themselves* to produce circa 1/32 new *strong bits* (ie. used get indexes into themselves to have the values there compared similarly to step 1d). Note that the relative order of both the selected affirmative and the untouched bits is claimed to be, essentially **random if the initial GMT was random** – so they should remain uniformly distributed after steps 1, 2 and 3.
4. Use the *untouched* bits to seed the main memory of a **Random-G Turing Machine** (name inspired from the independently invented Random-X algorithm used by Monero Blockchain [25]). Use the *strong bits* to key some changing 16 bit  $\rightarrow$  4 balanced bit truly random function (specified explicitly) to serve as *randomization translation function for Random-G emitted bits*. Use both the *selected* and the *strong* bits derived 1-3 above, to seed a new CSCSPRNG for the first Turing Round. The selected and/or strong bits COULD if desired be *decrypted* using a traditional fast cipher like AES-Serpent based on a portion of the round secret key derived as per step 0. This allows AES composability and further complicates (a bit more than mere obfuscation but still!) attacks on SKREMAAlpha even if the classical cipher like AES is breakable itself under mere plain-text attacks.
5. First Turing Round. Using the seeded Random-G Turing Machine as per Step 4, emit a sufficient amount of fresh *emitted bits*. When emitting a bit, use *randomization translation function f* constructed in step 4 above. Each time f is sampled, swap the sampled location randomly (using the CSCSPRNG) with another location of f, including itself. The Random-G Turing machine is essentially a multithreaded (sequential parallelism), self-modifying Turing Machine which performs simple operations based on its main memory. Each Thread has a “HEAD” in the main memory, and reads circa 32 bits of values there. Those 32 bits are swapped with 32 untouched once considered “from the end of the main memory” and they themselves are then replaced (in their new swap location) as per the below. Based on each of these 32 bit values and OPERATOR (from ADD, SUB, MUL, DIV, INDIRECT, JMP, JCMPIZ, FORK, CFORK, STORE, XOR, NEGATE, INVERTMODULO  $a+b^{-1} \bmod p$ , and others) and an optional OPERAND or OPERAND INDEX (in the main memory) of circa 16 bits (+/- relative to the HEAD position), an OPERATION is performed. The operation first uses the CSCSPRNG (provided for in Step 4, not the initial one) to *extend its OPERANDS*

- to 64-2048 bits (varies based on feasible performance.. typically will be 64 bits). This is done by using the CSCSPRNG in conjunction with its own or another *sponge function (Keccak-f is used)* to expand 16 bits into the required OPERAND size. Then, the OPERATION is trivially performed resulting another 64-2048 bit value, which is based through the sponge function again to get back 16 bits. Those 16 bits are passed through the dynamically changing *randomization translation function* (initialized in Step 4) to get 4 emitted bits. Additionally, those 16 bits are expanded to 32 and stored at the location from where the 32 bits were swapped into the current HEAD from “the end of the main memory”, overwriting what was there. A summary function of the computed 16 bit values (eg. a result of them being sequentially based through a sponge function) is stored as the **thread's nonce** (which changes over time with the operations progressing on said thread). NB: Some other similar strategies are being considered here, but in this variant the above suffice. Note that thanks to CMPZ, JMPs, FORKs and CFORKs, the threads (projected to be circa 1024-2048 at a time), will basically very quickly start to read from and overwrite each other's' used memory locations. So there is cross-thread “creative interference”. Additionally, after each full pass of 1 execution from each surviving thread, the new thread collection is ascertained for size. If more than ~2048 threads exists, it is cut down to ~1024, keeping only the threads where the *thread's nonce* is closest in absolute difference to the XOR of all existent thread nonces. The resulting surviving thread collection is Digulescian Permuted using entropy from the provided CSCSPRNG, thus changing the order in which subsequent passes execute threads. The execution continues until a sufficient number of **emitted bits are obtained** (resetting the main memory size to full, once exhausted or too small, circa 1-2 times)
6. The emitted bits from Step 5 are **enhanced**, undergoing roughly the same procedure as in Steps 2-3 (permutation and projecting over selected bits) – thus becoming uniformly distributed wfp and also being more strongly randomized and – indirectly - coupled with the bits of the secret key.
  7. The **enhanced emitted bits** from Step 6 are now used as a large sized secret key which is used to seed and initialize a fresh CSCSPRNG for the round, and Steps 0-3 are basically done again over a new GMT nonce (practically and untouched, protected second half of a 2 GB nonce GMT for example), to again, get circa 128-256 MB of **selected affirmative bits**, also circa 128-256 MB of **untouched bit locations** (fully untouched, located in the second half of the GMT) and some **strong bits**. The size of the enhanced emitted bits should be larger than the plain text size, so around circa 16 MB.
  8. The selected affirmative bits are used in some fashion (eg. simple XOR or via common sponge function passing) combined with the selected affirmative bits of Step 4. The obtained strong bits of Step 7, together with the strong bits outstanding from Step 4 and the *enhanced emitted bits of step 6* (circa 32 MB) are combined (eg. repeated XORing) with the *untouched bits of Step 5 (the then-current state of the main memory of the First Turing Machine)*. The result of the FIRST combination (the selected bits) is used as the initial main memory state of a new, second, Random-G Turing Machine, while the result of the SECOND combination, is used to initialize a new CSCSPRNG which supports it.
  9. The second Random-G Turing Machine is ran, until a sufficient number of emitted bits are produced. This should be on the order of circa 256 MB!! (so around the size of the outstanding **untouched bits** from Step 7). For practical purposes (memory/running time), it is also acceptable in practice to produce lesser number of bits, like 16-64 MB.
  10. The emitted bits as per Step 9 are enhanced (using the original 256 MB enhanced selected affirmative bits of Step 3 as a projection target) analogously to Step 6.
  11. The enhanced emitted bits of Step 10 are used as a seed to a “Digulescian Permuter” taking 5 sources of entropy (with the largest being this from Step 10) – which is a CSCSPRNG but may have a more basic and faster implementation than the one proposed below (given so much entropy to which it has access).
  12. The Digulescian Permuter from Step 11 is used to permute the globally **untouched bits** of Step 7 (so these are bits which were never read nor overwritten).
  13. Now based on the untouched bits resulting from the permutation of untouched bits, in Step 12, a **projection operation takes place** using the Selected Affirmative bits outstanding in Step 8 (initial configuration not after running the second Turing Machine on them) as a seed to yet another [fast] CSCSPRNG used which selects pairs of 2-bit value indexes in these globally untouched bits. This 2 bit value indexes are called locations. Now based on the comparison of the values there at, either a 1 or a 0 bit results or the values are equal. If a bit is emitted (probability 75%), then that bit should equal the required plain text bit. At decryption time this is guaranteed. At encryption, if the bit is the wrong value (when emitted), then **IN THE ORIGINAL GMT second half** the values at the respective locations are swapped. This generates circa 48 MB of encryption buffer. For added security pairs of TWO emitted bits are used to represent a bit of plain text (via their XORing). So this allows for encryption of up to 16 MB using 32 MB of the

encryption buffer. The outstanding 16 MB are used as the next block's secret key seed (SKS).

14. If this current block (so two-part GMT, like 2 GB) is *the first block (which is processed using a potentially short key – shorter than 16 MB)*, then the 16 MB of plain text buffer are in fact used as a “master key seed” (MKS) data which is required of all subsequent blocks.
15. Now if the plain text size (actual or future potential maximal size) is larger than 16 MB, more than two 2GB blocks are required. Encryption/Decryption proceeds sequentially over 16 MB-blocks of plain text, encrypting/decrypting into separate (unique, not reused, independent and truly randomly sampled nonce GMTs of circa 2 GB size – for both halves). The first block is called the **KEY GENERATION BLOCK (KGB)**.
16. For each subsequent 2 GB block (corresponding to 16 MB of plain text encryption potential), at ENCRYPTION time, a small portion (circa 16 MB) of the truly random nonce GMT are modified to reflect the proper plain text at decryption. DECRYPTION proceeds simply by taking the produced 16 MB corresponding to the plain text from Step 13 as the plain text. Alternatively, a XOR (or other combination) of the produced random bits with the plain text could be appended at the end of the blocks (and potentially be trailed by more truly random garbage to appear to be a multiple of 2 GB in size). But in its simplest (and current) incarnation, the plain text results directly from reading the corresponding “for plain-text” 16 MB portion of the encryption buffer of Step 13.
17. When a new block is processed, its initial key used for Step 0 will be a combination of the following: **the user defined secret key (0.5-16KB in length), the 16 MB MKS (derived from KGB), the 16 MB next block's secret key seed (SKS)** to obtain a single 16 MB private key input to step 0 (thus no longer being required to do key enlargement portion). The manner of combination is not critical so long as randomness is preserved. A simple recommendation is as follows: Using the 32 KB derived private key of step 0 of the 1<sup>st</sup> block, seed a “Digulescian Permuter” with small entropy, to permute the then-current state of the KGB. Then XOR the KGB with the SKS. The KGB for future round remains the modified KGB as per this step.
18. Output the data in the proper output streams / data structures. So for encryption, there will be like  $\lceil \text{size}/16\text{MB} \rceil + 1 \times 2 \text{ GB}$  the cipher text, using the current techniques.
19. IMPLEMENTATION DETAIL: Make sure to NOT copy secret data – user secret key, KGB, SKS or unedited nonces to any vulnerable environment (eg. don't just copy the second half of the GMT unmodified on the target [virtual disk] and just do the rewrites over it] – write it all from scratch in one go instead).

20. DONE.

Note that SKREMAAlpha can be used to encrypt arbitrary bitstreams and, when using simply to decrypt “000..” is claimed to be essentially a keyed probabilistic random function (KPRF).

If desired, before encryption, the plaintext could be enhanced with a truly random XOR mask (which is then published together with the encryption result). Or, conversely, SKREMAAlpha could be used in KPRF mode and the generated bitstream itself be used as the XOR mask.

There are a few “loosely-defined” concepts and procedures in the above 20 steps, which we clarify below.

### ComplexStateCSPRNG (CSCSPRNG)

This is one of the core components of SKREMAAlpha.

A CSCSPRNG is basically a CSPRNG which is enhanced with a few properties and behaviors:

- a) *Has the ability to ingest entropy on the fly.*
- b) *Encapsulates both confusion and diffusion by starting with a large universe (8k bits in this case), using it to get a smaller universe (2k bits in this case) - confusion, and growing back the latter into something larger thereafter (20k bits).*
- c) *Does a reasonably good job at bit-mixing.*
- d) *Propagates correlations and bit-mixing forward, entangling bits of initial seed with bits of new entropy its being fed. Can be considered a consequence of property a).*
- e) *Has a non-trivial structure and provides sufficient unsolvable confusion – even if its output were to be published as it (which is not).*
- f) *Has a sufficiently period before requiring injection of new entropy to keep its properties.*
- g) *Functions sufficiently fast to account for about 2-20 GB of bits generated using it per block (per 16 MB of plain text).*

Now here is how the CSCSPRNG in SKREMAAlpha is constructed:

- a) Initial seeded input (based on 32 KB + XORbuffer) is transformed into:
  - a. An ~8192+ bit good prime. A good prime is an RSA prime  $p$  where  $(p-1)/2$  is also a prime.
  - b. Initial parameters  $a, b, c, d$ , and  $e$  for three simple ModInvCSPRNG (MICSPRNGs) as per below. These are 5x8192 bits for the Large MICSPRNG and 5x2048 bits for the MediumWell MICSPRNG and 5x64 bits for the SmallFast MICSPRNG.
  - c. An initial state for 3 classical sponge function constructs for each of the MICSPRNGs. In

SKREMAAlpha Keccak-f [9] is used for this purpose. When the expression “is whitened” is used, it is meant to mean that some values of the input say in 1-4 bytes in size are passed through the Keccak-f construct, rotation happens, and then the same amount of bytes is squeezed back, followed by another rotation, and so on until all input is “whitened”. The term “to expand/extract” from the sponge function some amount of bits refers to the same process, but where a larger amount of bytes is used in squeeze step than in the push step (eg. 16 bytes squeezed for 4 bytes pushed for a 4x expansion). The variant of Keccak-f chosen is over 10240 bits (maximal size).

- b) **A simple ModInvCSPRNG** used here is basically a modular math based PRNG which functions as this. It takes parameters a,b,c,d,e and p as input (p being a good prime, the rest truly random). And does the following. It starts with a random initial state X initialized to 101010101... of size |p| (so ~8192 bits for the large MICSPRNG) and values project1 (circa 12-2048 bits – 12 in SKREMAAlpha) and project2 (initially set to 1010101...). Then, in order to be “squeezed” for bits it does the following: It rotates the secret internal state X according to this formula:

$$x\_new \leftarrow (a \cdot x^{17} + b \cdot x^{-1} + c \cdot x^{\text{project1}} + d \cdot x + e + 28)^{-1} + 11431 \bmod p \quad (1)$$

The generator spits “the middle” bits of X, ignoring the first and last ~512 bits (so the large MICSPRNG spits ~7k bits per operation) as emitted bits and rotates again. When new entropy is to be injected, it is processed as follows: **strong bits** allocated (as per terminology in Step 3) are distributed to project1 until saturated. The rest of the allocated bits are allocated to the first and last 512 bits of a,b,c,d,e after which these parameters are inverted modulo p. Whatever outstanding bits are considered XORed with the next positions in the XOR pool of the CSCSPRNG. The initial XOR buffer is understood to be roughly 16-32 MB large, being initially initialized to 101010101... whitened with the Fast sponge function.

- c) The CSCSPRNG proceeds in phases. For each phase it initially squeezes the Large MICSPRNG (for circa 7k bits in this case). The obtained bits are then whitened using the Large sponge function. Circa 1.312k of them are used to do a ROTATION round on the Medium and Small MICSPRNGS. Another 32 bytes x 3 are used to seed-refresh by overwriting the prefix 32 bytes of the sponge functions (so the opposite end from where push/squeezing happens), for a total of ~2k bits. This leaves circa 5k bits untouched.
- d) The CSCSPRNG proceeds to repeatedly squeeze a round of 1.5k bits from the MediumWell MICSPRNG a

number of times (which is 4 in SKREMAAlpha). These bits are then also whitened using the Medium sponge function. The 4x1.5k ~ 6000 bits are then expanded using the Small MICSPRNG (and its sponge function) to get 32 bits at a time from a 32 bits entropy injection a number of 4 times (6000/32 x 4 times in total to get 6000 x 4 bits). For each produced 32 bit value by the Fast CSPRNG, this value is expanded using the Medium Well sponge function into 64 bits (2x). The 5000 bits free from the usage of the Large CSPRNG are used to “complete” the 32bit values into 64 bit values when squeezing (circularly). So in total, a number of 6000/32 x 4 x 32 x 2 = 16 x 6000 = 96k bits are obtained at one time from a “grand” Large MICSPRNG rotation. These 96k bits are whitened using the Fast sponge function. Then ~6k of them are used as reseed buffer for the three MICSPRNGs (which will be XORed with incoming entropy pertaining to this), 32k are provided for the end-user of the CSCSPRNG, 26k are discarded (every other bit) and the outstanding 32k bits are XORed with the next locations in the XOR buffer.

- e) At consumption end, for each “big rotation” of the Large MICSPRNG, 16k bits are generated for consumption. They are XORed with the outstanding XORbuffer locations in the CSCSPRNG (wrapping around circularly on end), whitened again with the Fast sponge function and provided to the consumption component.
- f) Out of 32k bits generated, the system will produce in Step 2, back another 16k bits (roughly half) bits which are used to inject new entropy into the CSCSPRNG (which it distributes as per step a). Thus the production cycle is closed, consuming a grand phase for every 32k bits processed of the GMT.

A discussion is made in the next subsection of why certain choices were made and “why it needs / makes sense” for stuff to be so complicated and “how the hell use the algebraic ‘leaky’ Modular Inverse” here.

### **DigulescianRandomPermuter (DRP)**

This is a component which takes as input (1) some pool of entropy – divided into 1-5 semantic parts and (2) a list of indexes or an actual bit buffer AND permutes in place the list of indexes / the buffer (of bits, bytes or 2-bit locations).

It basically computes 1-5 distinct permutations (semantically only – they are not stored in memory) and applies them repeatedly in some sequence to the input (2) so as to permute it into a PRP for all intents and purposes.

Each individual permutation is basically a Schiffer shuffle like shuffle. This means that for each index of the (2) input, the current value (so index, byte, bit, etc.) is swapped with a value located in the (2) input at a location z which is uniformly distributed in choice from the outstanding ones.

The process is repeated until the required number of elements is processed / buffer is exhausted. Each  $z$  location  $z$  is chosen under some distribution transformation strategy based on the random bits produced by the associated provided CSCSPRNG (eg. ignore values outside the range 0..number of outstanding elements). For small added security – the shuffle is actually based on the cycle which the above sampling generates. Meaning the first element generated (say index 82 has its value stored in memory.. then the second element say at index 14 has its value replaced with that of index 82 and its own value stored in memory and so on).

The details of how the permutation works as follows. Consider the 1-5 permutations in order of increasing number of elements / entropy pool size:

- For  $i=1..5$ , Perform permutation  $i$ , on  $n[i]$  bits at a time, moving sequentially (eg. first the first 512 bits are permuted then the next 512 bits and so on).
- For  $i=1..5$ , Perform permutation  $i$ , on  $N/n[i]$  batches of bits (eg. a single permutation is performed moving around chunks of say 12 KB at a time).
- Repeat the above steps yet again, 1 time.

Note that this DigulescianRandomPermuter just permutes – it does not combine values, so it is called **nondestructive** (it neither reads nor writes to the actual content of what it permutes). Except when permuting the globally untouched locations from the second half of the GMT nonce, actually destructive combinations could be allowed, to get a **destructive** variant.

For purposes of SKREMAAlpha a destructive variant is considered unnecessary and out of scope. However, should the desire to quickly enhance SKREMAAlpha with it, once can think potentially of various tricks like:

- Keccak-f applied either over 16000 bit groups
- Keccak-f -like constructs applied over larger sequences of bits
- do casino-like shuffles based on input entropy and/or combinations with shuffled values
- consider the target input like a 3D matrix and do all sorts of rotations and operations on adjacent cells
- combine adjacent cells by doing modular inverse + whitening over a good sufficiently short prime (like 512 – 2048 bits, or maybe 8096 if super computer available)
- have some properties of bits of adjacent cells do something like the game of life Lifeseg cellular automaton (which is Turing complete in the general case btw)
- sort some columns / lines / sequences by taking either a classical hash of a portion of their own content which portion is thereafter eliminated (destructively)
- do modular inversion of feasible (eg. 2048 bit) portions of the target at a time and whiten
- ultimately contract the output if desired by some survivability rule
- XOR whatever is produced with the original input
- Repeat and combine steps in ideas above
- Ideas can flourish here far beyond current scope.

However, the security of Digulescian permuting arises from the fact the target of the permutation are actually ultimately uniformly distributed bits. So, the above, given the large incoming entropy pools, is considered sufficient to secure the correlation between input [entropy or initial user secret key on which it is based] and the bitstream given by the output.

### Random-G Turing Machine (RGTM)

This construct was sufficiently comprehensively described in Step 5. Note as a recap and expansion the following:

- The RGTM is initialized with a uniformly distributed random secret common memory pool for all threads
- OPERANDS are expanded from 16 bits to 64 before operations being performed.
- The result of all OPERATIONS is inverted modulo a 64 bit prime (fast) and the result whitened using the provided CSCSPRNG's Fast sponge function.
- Threads are pruned based on the Hamming distance from their internal nonce's to a combination of all their nonces. Threads are meant to overwrite and "corrupt" each other's memory space every once in often while so as to increase randomness.
- INDIRECT operands and generally memory references are also expanded from 16 bits to 32 using the Fast sponge function.
- Emitted bits are first passed through the randomization translation function (which maps 16 bits to 4 bits). After each use of the randomization function's space, the respective value say at  $f(x)$  is swapped with the value at a randomly chosen (using the provided CSCSPRNG)  $y$  from the  $2^{16}$  possible ones, including  $x$ .
- Emitted bits are whitened also.
- All operations are essentially  $O(1)$  and do involve some modular arithmetic modulo a 64 bit prime and whitening which is expected to be very fast.

### Selection Steps

To clarify fully, when the term select  $B$  bits from  $X$  based on source  $Y$  is referred to in this paper, the following is meant:

- a) By some method decide on a sequence or string  $S$  of  $2xB$  bits from  $X$  – either using a CSPRNG or elsehow anyhow, including sequentially -.
- b)  $Y$  is  $2xB$  bits long.
- c) For each bit of  $S$  in order, if the corresponding bit of  $Y$  is 1 select it affirmatively (the bit from  $S$ ), otherwise select it negatively (eg. becoming and *untouched* bit / location generally).

### Projection Steps

To clarify fully, when the term project  $Y$  over the target  $X$  to get  $\sim B$  **strong bits** is referred to in this paper, the following is meant:



- a) Until all the bits of Y are exhausted take a pair (a,b) of k bits (for example k=32 or k=4).
- b) Optionally using a CSPRNG or sponge function, or elsehow expand the k bits of each a and b to  $\log_2(|X|/2)$  bits.. so say like 32.
- c) Get the 2-bit value located index  $2*a$  in X as x and the 2-bit value located at index  $2*b$  in X as y.
- d) Swap the 2-bit value at indexes  $2*a$  with the last value in X and then do the same for b. Reduce the size of Y by 4.
- e) Compare  $x==y$ . If true skip.
- f) Compare  $x<y$ . If true, emit a logical **strong bit** of value 1.
- g) Compare  $x>y$ . If true, emit a logical **strong bit** of value 0.

### PrimeSqrtSPRNG (PSRNG).

This component is envisioned to be used when a particular relatively short bit sequence (for example the user-provided secret key) is needed to be expanded into something like  $2x - 4x$  its size with the extra portion hopefully reasonably unpredictable and hopefully uniformly distributed in principle (if the source is also uniform).

This works like this:

- a) Divide the input into “manageable” (in terms of extracting square root), but large enough consecutive sequences of k bits. So  $k \sim 128-512$ , with 128 for SKREM Alpha.
- b) For each element of the sequence a, compute the prime value  $x = 1 \text{ a r } 1$ , where r is incremented as a counter starting 0, 1, 01, 10, 11, 100, .., such that x is the first encountered **good prime**. As a reminder a **good prime** p is an RSA-like prime (the product of two primes of size  $\sim \sqrt{p}$ ) such that  $(p-1)/2$  is also a prime.
- c) Analogously to b) compute also  $y = 1 \sim a \text{ q } 1$ , where  $\sim a$  is the bitwise negation of a, and q another counter.
- d) Computer  $m = \text{sqrt}(x * 2^{(2*k+1)})$  and  $l = \text{sqrt}(y * 2^{(2*l+1)})$ .
- e) Use l to select k bits from m, and emit them.
- f) For all emitted bits (so Y in total), reverse their order.
- g) Then intertwine them with the bits of Y.

### Of primes and modulo

The selection of a good prime based on a seed a like this:

- a) The value  $x = 1 \text{ a r } 1$  is computed, where a is the seed and r is counter as in point b) above, such that x is a valid prime number.
- b) Analogously compute  $y = 1 \sim a \text{ q } 1$ , so that y is a valid prime.
- c) Check if  $p = x*y+1$  is an RSA-like prime.
- d) Check if  $(p-1)/2$  is a prime.
- e) Prime checking is done using probabilistic prime checking with a small base sequence like 2, 14, 17, 18, 20, 23, -25, -26, etc., so that the process is

reasonably fast. Actually, prime construction is currently envisioned only in step 0 of the first block. So even if takes a reasonable while its ok (by comparison to how long everything else will take).

## IV. DISCUSSION OF SKREMALPHA

### A. Discussion

SKREMAAlpha does have a rather large number of stages and looks rather complicatedish.

The core idea is like this: *Initially the quality in terms of security (unpredictability) of input-output correlations cannot be proven to be high, but as the Steps progress this will quickly jump up to being fully secure.*

Now the output of early stages is not published – it is not available to the attacker. Only the final result in Step 19 is visible.

The Boolean Circuit Complexity (bit mixing) is claimed to rapidly increases in Step 1, thanks to the “feedback loop” where the selected bits (which are not published) are in part used to reseed the CSCSPRNG, thus (i) making it operate on  $O(\text{GMT})$  entropy for later stages and (ii) propagating early bit dependencies [of secret key + GMT] forward over a rather large number of reseeds. Furthermore, the selected bits are projected over the GMT to get uniformly distributed random bits which are also used in the reseed. The reseed itself is based on Modular Inverse of a short polynomial over a large prime (8096 bits) and whitening using Keccak-f.

The functioning of the CSCSPRNG is intended to ensure *the a good bit mixing of incoming injected entropy with exsistant secret pool*. This is achieved using the repeated modular inversion + constant shift of a short polynomial over a large prime (8096 bits). Confusion is also achieved by “projecting down” this value to 2048 bits. The final whitening step is basically considered an added “obfuscation” which is intended to further degrade and destroy algebraic correlations. So for all its criticism for being slow, and “algebraically leaky”, this procedure is claimed to be secure because its results are not published directly: even for Step 1, the later stages of selected bits will depend on sophisticated combinations of secret key bits (over a fixed nonce) since a lot of entropy will have been imported and will have been reasonably well mixed into the secret state. Furthermore, to the knowledge and confidence of the author modular inversion (much moreso over a nice polynomial not a linear combination) is the only good bit mixing function for a large number of bits (thousands to millions at a time) he knows. So it was desired to mix a large number of bits at a time – 16 million was desired initially, but for performance purposes this was cut down to 8096. Also, modular inversion produces a large number of bit changes (circa 50%) in the target. So, in the current construct where entropy is rather quickly refreshed, we found this adequate. Considered and rejected for security weakness were: (i) AES in any mode; (ii) Keccak-f whitening; (iii) implementing another Keccak-f like function ourselves over a larger number of bits; (iv) the Chaos Machine of the initial SKREM-variant (which is also much slower) since this was

credibly determined to be insecure as per Section 6 – most likely because of problems with attractors in chaotic systems of the used time which lead to successful similar keys -like attacks.

The polynomial construct used in the ModInvCSPRNG is furthermore intended to add unpredictability in input-output correlations. Notice the short-entropy random exponent in the power for the factor  $c$ . Notice also the  $x^{-1}$  exponent which gives a large degree to the polynomial. The shift values 28 and 11431 was chosen to be an arbitrary positive representative beautiful constant which is intended to break algebraic structure a bit and introduce unpredictability further. For performance purposes this was made a constant, not another coefficient like the  $a$ - $e$ . Based on simulations performed with ChatGPT v4o, on ultra small sized versions of such polynomial + inversion functions (on very small primes), it was found that the collision rate is around ~43%. This means we expect that the function is NOT bijective, but instead chooses relatively uniformly at random like from 57% of its potential universe. Over many iterations this actually is intended to help in terms of confusion. Working back from the end, an attacker would have to hypothesize like a base ~2 exponential branching space. This particular construct was actually chosen based on both personal author Codeforces.com experience [26] and Knuth Volume 3 [27] experience with creating PRNGs. Since fresh entropy is injected rather frequently and in large amounts, we did not insist much on strict uniform distribution enforcement (we already get a lot here from the modular inverse operation) nor on complete and absolute lack of algebraic structure of a single step in the transformation.

Based on lessons learned, after just Step 1, we do not feel confident enough that the security of the produced result is sufficient to reliably defend against AI/ML based attacks as is. However, the results of Selection are never published as is directly, but instead used for subsequent steps.

The *Digulescian Permuter* is essentially used as a non-destructive bit mixing function over a very large pool of bits (MB to GB). So for the processing of the second half of the GMT, the untouched locations actually need to be left untouched – since they will be modified to encode the plain text, their values cannot be overwritten or considered before the final stage where the text is injected. Also, even for the first half, for MB-GB sized targets, modular inverse is simply too expensive in terms of running time to be usable. It is based on the Schiffer shuffle so given enough entropy it is known to produce uniform probabilistically random permutation (PRP). Note also that at this stage of Digulescian Permutation the original secret key comes again into play (it generates a short like 1024-2048 elements permutation which is one of the parameters). This further introduces unpredictability thanks to lack of knowledge of the private key. The Digulescian Permuter could be further enhanced with ideas on it presented in Section 2 – however we feel at the moment what was given suffices.

Again, for intermediate stages on the first half of the GMT, the aim is basically to import entropy in as unpredictable manner as possible, introducing confusion and

diffusion – we do not claim that if their individual outputs were published directly that they were sufficiently secure against AI/ML based attacks: although even individually Step 1 and Step 5 look pretty strong.

Note that we at times enforce and require uniform distribution and added security quality of some bits

The novel piece-du-la-resistance introduced in SKREMAAlpha are the Turing steps. So the Random-G Turing Machine introduced is conjectured to produce nearly random output, being computationally unfeasible to determine an input to produce a certain output. Furthermore note that a generalized single-thread version of it is Turing Complete: thus an arbitrary Turing Machine could be encoded there in the first half untouched bits. Determining if an arbitrary Turing Machine in the general case can produce a certain output on some input, is considered a very hard problem, equivalent to Halting Problem. Now RGTM works on finite memory and running time, so its not a general Turing Machine.. but still the restriction to finite tape Turing Machine makes the general case something like PSPACE-complete which is meeh.. nice. The RGTM is basically a set of rewrite rules (the OPERATIONS) which is set loose on random memory. Note that Random-G was invented independently of Monero's Random-X [25], which was discovered during literary research much later. However the success of Random-X as a proof-of-work construct hint positively for Random-G. While Random-X is not intended as a keyed PRNG (KCSPRNG) its success hints that for such constructs, even predicting its output is as hard as running the program itself.. and finding a feasible input for some output is very hard.

The Random-G Turing Machine is conjectured to produce uniformly distributed secure random output (when fed with quality entropy the way it is done in SKREMAAlpha). In fact in simulations ran with ChatGPT v4o, ran on even weak entropy sources (C# Random class) the construct passed simplified NIST randomness tests and had near-perfect stats like entropy and correlations. Similar keys was not tested. The key was doing modular inverse (even without whitening) on the result of each OPERATION. So RGTM can be viewed, in this construct, as a keyed cryptographically secure PRNG (KCSPRNG). And in fact, so can SKREMAAlpha be used as such overall.

Because RGTM might not produce fully uniform distribution, and also in order to even further improve security, its outputs are also not used directly, but Digulescian-Permuted and then also projected over a truly random table (which the selected bits even transformed nondestructively are).

Going into Step 7, the secret pool is already 16MB+ in size.. so plain-text sized. Now this secret pool is sort of “midways” in the execution of SKREMAAlpha. Meaning it is conjectured as unfeasible to be characterized or even have predictions about it from either endpoints (starting with partial predictions of user secret key or with knowledge of final output of SKREM – known/chosen plain text).

### B. Statements and assumptions

Here are the core assumptions, usage pattern, and important technical decisions in SKREMA $\alpha$ .

1. The expanded original secret key is secret and truly random.
2. The untouched bits in both first and second half are not read nor overwritten in anyway before being used.
3. The size of selection window for Steps 1 should be as large as performance allows – in this case 2048 bits at a time from 32KB. This should be above the ability of the attacker to precompute or cryptanalyze.
4. The size of the large prime chosen in the CSCSPRNG should be as large as performance allows. In this case 8192 bits. Choosing directly just 256 bits or even 1024 bits is considered worrisome. The size of the medium prime should be circa 1/4 of the large prime. Finally, the size of the small prime should be small enough to allow CPU registry operations on it – so 64 bits for 64 bit systems.
5. Injecting entropy by overriding start and end of X in the MovInvCSPRNG is relevant. Practice shows that small changes in prefix/suffix produce near 50% bit changes in result.
6. The injected entropy in the MovInvCSPRNG should be large enough so as to be beyond the attackers ability to cryptanalyze.. so 512 bits here per iteration is considered sufficient given that the original secret key itself is 16 KB and overall there will be circa 1 million reseeds overall of the CSCSPRNG.
7. Threads are meant to interact and overwrite each other's memory space in the RGTm.
8. The set of OPERATORS of the RGTm are intended to make the process act as much as possible like a Random Circuit – which, according to ChatGPT 5.0 is known to have excellent cryptographic security properties, when truly random (so large and good quality entropy) and sufficiently large and of the proper type (intertwine of constant fan/fanout, balanced random function gated layers). Basically uninvertible and worse!.. generally an input exists for most output restrictions (so for example guessing half the output is not informative since anyway you would guess it, the result is that “yes, there exists some input producing it”). So great confusion and diffusion.
9. It is important that operations of the RGTm happen on a large number of bits 64-2048.. ideally beyond the cryptanalysis capability of the adversary. 64 was chosen here for CPU performance reasons. It is acceptable to use just a small seed (16-32 bits) in the main memory and expand it using CSCSPRNG into 64. For performance purposes, in extremis, even mere whitening should suffice for the first Turing step (but not the second). However, it were great if OPERATIONS could be seeded using 128-256 bits and occur on 8192 bits or more :).. however at the current time, such constructs were too impractical.
10. It is considered insufficient to use just the first half enhanced emitted bits directly, without doing the rests of the steps.
11. Whitening function (Keccak-f used here) should be fast and produce reasonable obfuscation and unpredictability. It is not considered nor required to be secure in itself. However, when used in conjunction with other CSPRNG techniques, it is useful to (1) break algebraic structure or any sort of systemic general traits of a CSPRNG approach unrelated to the whitening function technique itself and (2) improve uniform distribution a bit.
12. The sampling of the next element in the individual permuter of a single permutation in the Digulescian Permuter is supposed to be provided with a lot of entropy (ideal all the entropy it needs to produce the full permutation without reuse). The distribution conversion strategy (from uniform to whatever is needed at that particular moment) does not need to be something fancy or perfectly uniform. However, simple rejection sampling can be useful to further complicate the analysis of the internal state of the CSCSPRNGs.
13. Projections (and thus strong bits) are considered an important element of SKREMA $\alpha$  and should not be skipped. The quality of the strong bits – especially at early steps – is “felt” to be much better than that of other bits (in terms of security/unpredictability as well as of uniform distribution).
14. The usefulness of the PrimeSqrtPRNG is based on the conjecture that the square root bits of most primes are normal in any base (including 2) and can be considered uniformly random and generally unpredictable in terms of pattern characterization (so if input is unknown, there is nothing which can be stated about the output). They are not used this often because of the performance penalty of  $O(n^2)$  for taking the square root up to  $n$  bits. However at places like in the expansion step of the CSCSPRNG, SKREMA $\alpha$  could be enhanced with expanding values using PSPRNGs.
15. As discussed in subsequent sections, we consider a truly random secret key of size 16 MB (yes, MB not KB) to be what reliably provides security for ANY deterministic cipher until at least the end of the 21st century. Also, the effective bits of unknown input required for any cryptographic circuit generating algorithm – which also lower bounds the user key size – is estimated to be at least 16KB. Unfortunately 16KB is already a bit too large to store in the human's memory efficiently and reliably. So step 0 key expansion was precisely used to build up until 16 KB in a manner which should be sufficiently secure in practice for at least a long while. Then, the buildup to 16 MB is achieved using the Key Generation Block (which is also plain text size, so information theoretically there should be enough entropy for perfect security if SKREMA $\alpha$  were

bijjective). This method is considered secure, because essentially the attacker only knows (ideally) the plain-text so he can conjecture, hypothesize or test properties of the 16 MB plain-text block original key.. which is, by construct truly random (consists of arbitrary bits, in fact they can be chosen arbitrarily and independently for each block at encryption time.. so yeah.. “for the love of all cats, may peace prevail on Earth ..” goes also). Now given some correlations or speculation/hypothesis about 16 MB truly random bit sequence, asking “is there any input to the KGB which produces these correlation” is like, generally, asking “is this random output producible by a random input, on this 2GB sized circuit which is intended to act as a random circuit?” – so the attacker is for practical purposes denied the advantage of actually knowing the plain text.. he is tasks with solving random\_output --> random\_input mappings for the KGB.

16. Ideally the user provided key should also be generated truly random and not be chosen as a passphrase or the like by the user. However, given the sophistication of the KGB, for practical purposes a passphrase + a public nonce for the key is acceptable.
17. GMT nonces are unique, truly random and not reused across encryptions.
18. SKREMAAlpha is claimed to be secure to use for streams of plain-text up beyond 16 MB, so basically unbounded in size. Note that nothing in the functioning of SKREMAAlpha depends on the plain text except the final swaps of locations. So basically data about plaintext does not affect subsequent nor prior block rounds at all. Thus, the problem with “hash patterns” with AES when used in any mode [28] [29] is not one of SKREMAAlpha.
19. Now on the breakability of SKREMAAlpha, it is very difficult to claim an exact bit-security. In Section 5 and 6 we discuss some modern attack vectors, and we claim SKREMAAlpha is secure against them even for NSA-/state grade actors with massive preprocessing and compute power. For this a 16 KB key is required. For common usage patterns a 512 bit key suffices. By unbreakable we mean that even if all the NSA datacenter did preprocessing to break SKREMAAlpha for 10 years, then once presented with the plain text spent another 10 years to break it, it wouldn’t succeed whp, regardless of plaintext. Furthermore, by breaking, we mean here “distinguish a SKREMAAlpha encrypted output from random, even known [attacker chosen] plaintext”.
20. Note that after the experience with the insecurity of SKREM due to AI/ML based attacks, the security claim in point 19 above is like **“if anything which is not computer-aided design is secure, then SKREMAAlpha is secure”**. Also, it also has the caveat **“or something very close conceptually to SKREMAAlpha – so SKREMAAlpha with minor modifications”**. So somewhat in the sense of “if

**keyed probabilistic random functions (KPRF) exist and one admits a simple design which is short enough to fully fit like 100 pages of scientific paper, then SKREMAAlpha is one of them.”** This is as opposed to one being found exclusively via automated tools AI/ML or as the conjunction of automated tools and human researcher (similar to solving NP-Complete and other hard problems as discussed in [29][30]). The reader and the public should take the above statements as an indicator for strong confidence in the security of SKREMAAlpha. Contrast it with the total lack of confidence the author has, justifiably, in NIST primitives like AES.

21. Note also that the large circuit size proportional to  $2^{68}$  using classical  $t^2$  transformation complicates the practicability of using a quantum computer to attack it.

Now here is what we assume the attacker CANNOT do. If too many of claims below fail, and fail badly, that would compromise the veracity of our unbreakability claim:

- The ModInvCSPRNG used in conjunction with the entropy refresh mechanism in SKREM produces such an effect that, after a few (say 16-32) iterations of refresh, key recovery attacks (original state of the MovInvCSPRNG) are not possible.
- The Random-G Turing Machine (RGTM), initialized with truly random memory data acts as if it was a random-circuit in terms of cryptographic performance. In particular: (1) given its full output or part thereof, computing the input is not possible; (2) given a partial input of the memory config [but not the CSCSPRNG’s state], computing if a given output CAN BE produced by that input portion is not possible; (3) the output produced cannot be discriminated from random noise by the adversary (without knowledge of secrets ofc).
- The DigulescianRandomPermuter mixes the order of bits sufficiently well so that (assuming sufficient large quality entropy is provided), an adversary without access to its entropy source, even when given the input target, cannot feasibly (1) determine if a given output was [likely] produced by a DigulescianRandomPermuter applied to the input; (2) discriminate a given output from a random permutation / bitstream. Also, given the output, and a partial input candidate, but not the entropy source, an adversary cannot feasibly discriminate an entropy configuration exists or not which can map some input which satisfies the (short) input pattern to obtain the output.
- At stage 19, the permutation of the outstanding untouched locations is for all intents and purposes undiscernible by the adversary from a true random permutation. Note the the adversary does NOT get to see the permutation itself, but rather the bit sequence that its processing (comparing  $v[\text{left}] < v[\text{right}]$ ) produces. So, the adversary sees even less than the permutation – however, even if he were to see the

permutation itself, it would look for practical purposes truly random to him. In particular the probability of having the index of *left* be HIGHER than the index of *right* or vice-versa should be very close to 50% over any GMT and private key whp. Similarly, the probability of having left or right from the first/second half of the second half of the GMT should analogously be close to 50%.

- Inverting the Turing Machine Step 2 given some output is for all practical purposes same as asking “Is there a <TM, input> pair which produces this output?” and should have both (1) a very large (exponential) number of positive answers; (2) percentage wise a very small number of positives (like less than  $2^{-1024}$  probability). This should hold whp for any not trivially short, but not too long partial output. Even given a full output, determining the <input, CSCSPRNG> pair used to produce it is not possible for an adversary.
- The final enhancement step over the emitted bits of the second Turing Step actually adds like-wise entropy to the result: meaning roughly that for a given final output, and any Turing Machine used in the second Turing Step, there is generally some configuration of the entropy used to enhance the emitted bits (Digulescian Permute, project) which produces the output – like more often than  $2^{-1024}$  or so. Also finding even one exact match should be like less often than  $2^{-1024}$ .
- **So, the last three assumptions/conjectures above, taken together, assuming also reasonable qualitative key derivation in step 0, serve to final claim that SKREMAAlpha is actually a KPRF (Keyed Probabilistic Random Function) to such an adversary. And that any meaningful characterization, key recovery attempt, or even attempt to distinguish from output (even assuming known or chosen plain text) is computationally impossible for such an adversary.** Note that this assumes GMT nonces and user chosen private keys are chosen truly randomly, are not reused and are secret.
- In terms of compute security power, we expect that if SKREMAAlpha is secure today (year 2026), which we claim it to be, then it will be secure throughout advances in computing over the entire 21<sup>st</sup> century – including quantum computers, except potentially some computations based on new physical principals (eg. Unity Device -like systems, Accelerated Turing Machines, etc.). So we expect like any adversary even at the end of the 21<sup>st</sup> century, with circa 75 years of precomputation on SKREMAAlpha system, should take at least 100 years or more to break even a single SKREMAAlpha cipher instance. So in terms of compute power... for the final break step, we claim security of above  $2^{256}$ . For current systems, we expect attacks on the private key (weak user chosen key / generation process) to actually be the only thing which might in practice attack SKREM

(which is actually a misuse pattern, not a breach of claims). So, for current systems and know-how, we take like this: 10 minutes for solving a 1024-bit SAT instance on a super-computer (note as per Section 7 that there is probably no transformation to actual SAT circuit of encryption algos for breaking them by state actors currently – not required). We assume  $n \cdot \log^2 n$  best case growth to larger number of variables. We apply this over the  $\sim 2^{34}$  variables of SKREM. So the factor is  $2^{24} \cdot 1156/100 > 2^{27}$ . Multiplying that by 10 minutes gives  $\sim 2500$  years. And the above is like is SKREMAAlpha in terms of internal complexity of circuit is basically almost as weak as a random 3.7 term randomly built, forced satisfiable SAT instance. With advances in photonic computers and some variant of Moore’s law.. and more importantly, advances in AI/ML models, we still expect SKREMAAlpha to offer 100+ years of security in the year 2099 if it offers 2500 years today. It is unlikely that quantum computer of the current models (based on qubits or on photosynthesis like fast path computations) will be used to attack SKREMAAlpha this century (cost of constructing circuit / time of running the experiment repeatedly, versus the value and expiry timeline of whatever information is hidden by SKREMAAlpha).

Ultimately the following claim is the strongest we can make soundly:

**“If any exclusively man-made [without computer aid] symmetric key cipher is secure then SKREMAAlpha is also secure under whatever model the first cipher is.”**

So, it’s like saying SKREMAAlpha is as good as anything which is as good as it gets so long good enough exists.

If the above were to NOT hold, then either of the following or their conjunctions would need to be used:

- **AI/ML models are used in conjunction with the human researcher AND prospective attack systems (datacenter-sized attack simulators) to produce specific uniform / even non-uniform ciphers which have a program size + hint which goes like in the MB to GB.** I recommend advanced agentic AI based on non-neural network genetic-algorithm-like non-uniform learning algorithms here. Note that SKREMAAlpha is a uniform constant, small sized algorithm, which just makes use of the RTM (Random Turing Machine Model). So the truly random GMT nonces are NOT considered to be part of the algorithm – they are just random bits sampled at running time. But with the above ideas, combined with others such as those floated in our prior paper “Towards Solving NP-complete and other hard problems efficiently in practice” [29][30], an actual C# program size of like 10 MB – 1 GB is expected. Conversely, generating a “good nonce” may also be an intermediate computation step, just like when RSA was a thing, generating a “good” RSA prime modulo was a step. Note that, according to HUMINT, the AI/ML adversarial model approach is

currently used by Western Block specialized agencies to producing “secure ciphers” (which their current systems cannot break). We, in the longer run of the 21<sup>st</sup> century consider mere AI/ML adversarial approaches based on current learning algorithms and attacker algorithms as INSUFFICIENT to generate secure ciphers. Use SKREMAAlpha then augment it with adversarial AI/ML – yes – good. Simply make a neural network model as an encryptor based on counter-adversarial AI/ML learning – no. Also -no to just making some primitives like mixing function exclusively based on adversarial AI/ML while still keeping their block sizes and running times very low – this also – no.

- **A good ML agentic crypto generator based on a self-modifying, ultraparallel, random, non-uniform learning/acting algorithm as envisioned by us roughly now.** Note that such an agentic learning algorithm is intended to be general purpose – may or may not form the object of a future paper, and anyhow is expected to require decades to train properly to generate anything meaningfully useful.
- Use the tricks hinted on in Section 7 to further deny visibility to the output of SKREMAAlpha to the attacker. Those tricks involve Turing Complete rewrite rules-based games and mathematical theories beyond ZFC.

Note that post 2050, we expect attacks based new physical principles (so what is considered largely non-physical by mainstream science in 2026, so definitely beyond the standard model) to be able to break any “short” SAT circuit in like polylinear time (except experiment hardware construction which we expect asymptotically to also be polylinear in time and resources).

### C. *A word on lack of formal security proof*

Experience and HUMINT have convinced the author that in the post AI/ML era, formal security proofs by reduction from an “known” security primitive or combination thereof are essentially meaningless.

Most proofs in cryptography pertain to reducing solving the cipher attack (for a particular attack) to breaking the security of an *assumed or conjectured “safe”* primitive, or its security promises. So basically, they involved reducing from either “breaking AES” or “breaking Feistel-networks” or “prime factoring” and so on.

The only relevant information-theoretical proof is the domino construction for a 1-way function candidate which is indeed a 1-way function iff such exists. Which is not used in practice.

We the Author, in this paper hereby affirm and conjecture that ALL of the classical “simple” cryptographic primitives, including AES, Serpent, Feistel-networks in general, even Keccak, ARE vulnerable to distinguishing keys attacks driven by AI/ML automatic solvers, for a datacenter-large adversary with enough precomputation time in less than 20-30 minutes (state actors

mainly). This applies to all NIST ciphers and primitives of which the author is aware and also to Russian ciphers like GOST and basically any “simple and short” cipher. Simple and short it is a rather funny concept but it is understood to definitively include “any cipher with a short block size (like 256-1024), a short key (like 256-1024) and simple, fixed, short transformations which are applied repeatedly in rounds”.

While not a “short and simple cipher” itself, the fact the original SKREM variant was implemented and found insecure by the RAND so quickly after being published (2022 vs 2021), further supports the above claim.

This exact “everything now is breakable” claim is elaborated upon in Section 7.

Notwithstanding the above, the way in which AES and Keccak-f are used in SKREMAAlpha look most certainly to be able to allow a security proof by reduction from them, as to the unpredictability of input from output AND of generation of a specific output based on [partly] free input. However, due to the above claim of insecurity of basic primitives, we find no value in investing time, effort and paper space in formalizing such a proof.

Ultimately, the security of all present-day ciphers’ relays on “how good a SAT circuit” they construct. In this regard, SKREMAAlpha shines in the following ways:

- It’s induced circuit size, taken as the square of its running time, is thus proportional to the square of the size of the GMT public nonce (per block) so about  $(2\text{ GB})^2 \sim 2^{68}$ . Oppose this to like  $(24\text{ rounds} \times 256\text{ stepsish per round})^2 < 2^{26}$  for AES.
- Furthermore, each encryption has a different circuit in terms of the secret key (each claimed to be secure whp), from a family of nonce-sized strings, so of size  $2^{2\text{GB}} \sim 2^{16000000000}$ . Contrast this with the fixed for all runs families of circuits of all NIST primitives currently, including AES – so their circuit is from a family of  $2^0$  circuits or  $2^{256}$  if using IV at best.
- Most importantly, the sizes themselves can mean nothing if the circuit used is weak. “How sophisticated and hard to break is the induced circuit” is what counts. In this sense, the use of the Turing Machine Step in SKREMAAlpha (twice) is conjectured to make the end-result it produces (especially after projection over truly random data) essentially have the same traits of that of a **randomly built circuit of proper configuration (AND, OR, XOR layers stacked)**. Empirical evidence in literature discovered with ChatGPT v5.0 claims that such random circuits produce outputs indistinguishable from random and that furthermore, they are even information theoretically not invertible. Furthermore, the construction of **gradual** incorporation of entropy from the GMT based on **both** the entropy thus far incorporated and the secret key (as opposed to just secret key in the original variant of SKREM), done with a reasonably secure bit-mixing function approach (versus no bit mixing

in the original variant of SKREM) is intended to de-facto make the actual input for the Turing stages inaccessible to the attacker.

Ultimately, we take note that any actual and correct proof of the security of any cipher would entail a proof of the irreversibility of its circuit. This itself would hint that “at least some instances of SAT problems are hard”, ultimately giving a negative to the problem of  $P=NP$ . However, even if  $P=NP$ , but like the probabilistic average case for solving an NP-complete problem of the likes induced by SKREMA $\alpha$  is say  $O(n^2)$  in circuit size, AND if the  $O(n^2)$  circuit construction is a lower bound for any circuit [both of which are unknowns], then breaking SKREMA $\alpha$  were still  $O(n^4)$  so proportional to circa  $2^{136}$ . If  $P=NP$  and solving SKREMA $\alpha$  took  $O(n^3)$  in circuit size, then it would be proportional to  $2^{204}$ . Contrast this, with the case for AES when the values would stand at  $2^{52}$  and  $2^{78}$  respectively. Anyways.

Finally note that SKREMA $\alpha$  is fully composable with any classical cipher. Besides the optional AES-decrypt before the first Turing step, one could do either or both of:

- (a) classical encryption/decryption of the full GMT
- (b) classical encryption/decryption of the plain text.

Choice (a) was claimed by the HUMINT source exfiltrated in 2022 to be secure even for the original SKREM variant.

For composability of security claims, if either of choices (a) and (b) are done, they need to be done so with independent keys from that used for SKREMA $\alpha$ .

#### D. Realizability and performance

SKREMA $\alpha$  was designed as intended to operate on a higher-end commodity hardware as of 2025 in like 20-60 minutes per block. So this make like encryption of a movie clip with sound something like an hour or a working days' worth of computer time.

The most expensive operation in SKREM is the modular inverse mod a 8096 bit prime. This is done roughly circa  $2^{20}$  times per 2GB block, give or take. So maybe around  $3 \times 2^{10}$  seconds for that so 1 hour on a laptop. Maybe 2 with everything else going on. This will be known when SKREMA $\alpha$  is fully implemented and tested.

In terms of memory usage, implemented carefully, using some basic tricks, it can be made to consume just 2-4GB, given access to I/O streams for [re-]reading GMT and key and plaintext and writing output where appropriate.

The current state of the implementation by the author, nevertheless is sufficiently advanced to show to us that, at the very minimum, SKREMA $\alpha$  is realizable fully.

Additionally, some relaxation on some parameters could be done in-extremis to accommodate reasonable running times (the most obvious and painful one was to cut the large prime from 8096 to 4096 bits).

#### E. Sidechannels and other implementation details

The scope of this paper targeted only providing a secure, claimed unbreakable symmetric key cipher / keyed probabilistic random function (KPRF). Nevertheless we acknowledge the fact that, in practice, given the actual construct of SKREMA $\alpha$  care needs to be taken as to side-channels.

As an “in a nutshell” advice – as of 2026 (and post 2020 actually), for “encryption to work” especially but not limited to when using commodity hardware (ie. hardware bought off-the-shelf in the Western Block), the following holds:

**“Encryption/Decryption and management of secrets MUST be done exclusively in BUNKER MODE, in an air-gapped and fully isolated manner, and all hardware physically wiped and destroyed thereafter”.** Bunker mode is understood to mean like in a Faraday cage where no EM emissions are possible neither from inside nor from outside into it. All devices must be within the bunker and no leakage is allowed. In terms of air-gapped means that no machine ingested in its setup any data from the Internet except what was clearly under the control and vetted reliably by the actor. Isolated manner means that I/O between the Bunker devices and the outside world – in whichever form must happen free of any untrusted hardware (not even a photo camera or SD card), preferably via human memory and a keyboard (on the outside world). By physically wiped it is understood to mean all components including like all motherboard circuits and any peripherals used to the point of for example them being fully liquified by molten lava or a welding phaser (over 1800 degrees), or pulverized: all while still within the bunker, not taking them “offline” outside. Beware of downstream “backdoor-activations” when a device which had become compromised, via a specially crafted bitstring (carried like over an infected SD drive) compromises another device and makes it go “zombie”. Especially mean in the context of how data is bought “into bunker mode”. Great care! Air-gapped is insufficient.

While out of scope we do present a few basic ideas on practical implementations and usages of SKREMA $\alpha$  and for that matter, of most or all ciphers:

- Supply chain matters. Remember the word Apollo from the 2014-2024 Israeli operation which got exploding pagers into the hands of most of the fighters and leaders of its foes.
- Encryption/Decryption should be done on a machine where no extraneous software runs. When used in the Western Block for example, a Chinese market bought system, with a trimmed down variant of fully open source Linux running on it – preferably single-threaded is the recommended environment. I/O of data should be done via SD / microSD cards preferably used via an PCI port not USB. Once data transfer is complete SD / microSD cards should be melted with a welding torch until liquified.
- If any kind of software runs concurrently on the device during encryption/decryption OR if the hardware comes from untrusted supply chain, NO MATTER WHAT precautions are taken, side-

channels are not preventable. Especially with AI/ML powered analysis RAM access delays deltas can be used to deduce with high accuracy CPU activity. Additionally, private stuff can be stored in firmware chips (eg. USB interface) or sent over WiFi/Bluetooth adaptors even if they appear fully offline. Also anything with physical direct, or EM indirect access perhaps (eg. discombulator) to the electronics of the device can in principle make RAM reads. Anything which even in part, even while shipping in sealed packages “touched” a western block jurisdiction or their satellites is considered untrusted in this context.

- Isolated Enclaves, VM machines, Trusted Boot, BIOS battery removal (for systems newer than 2020 at the very least), Trusted Computing, using peripherals to store and compute (eg. GPU), nothing of this does not help against supply chain and/or side-channel attacks not in the slightest. The only thing to reliably help is full EM isolation of the encryption/decryption device while operating AND secure I/O data transfers AND physical wipe of the encryption/decryption machine when BUNKER mode is no longer sustainable.
- Beyond this, consider that SSD are considered impossible to wipe by software reliably. HDDs – older models (before 2010) can be wiped if careful trivially with truly random 10x rewrites – if cache is also wiped. Still better to just melt / pulverize them and burn the pulver.
- As elementary precautions so as to basically at least make things a bit harder for lower-grade actors, consider doing the following: Full memory wipe after encrypt-decrypt (so not just hardware reset, but software rewrite). Hardware RAM reset, followed by at least 10 second cooloff time, followed by software wipe of RAM again.
- Secrets should be imported/exported via physically destroyable media such as microSD cards. Encryption/Decryption machines should not have access to large writable durable storage. Western supply chain unfortunately still offers any off-the-shelf currently, as per empirical brief research, still a few MB to GB of storage via component’s firmware.
- There is no way to reliably verify a hardware component (eg. CPU) coming from an (even partly) untrusted supply chain if it is secure. For example, repeated signals of a particular pattern could be used to make capacitor trigger some physical aspect which acts like a SWITCH replacing the “path to” the firmware. No empirical nor any reliable microscopical way to verify. The only verifiable limit is to take “minimum transistor separation distance in micrometers as per state of the art technology” and divide the volume of whatever device there is by that value to get a rough limit on MB of storage capacity. In fact not even that.. storage may not use transistor but simpler or other

technology, potentially also interactive with environment of the hosting bunker.

- Beware that Windows OS generally writes directly to durable store filesystem. So for example if you have a virtually mounted drive Z:\ which is mapped from an encrypted file “C:\alabala.srt”, then occasionally Windows will overwrite the filesystem of this C: file without carrying about the drivers using it (for example access times, etc.). Beware also of NTFS alternate file streams. Also since with Windows 11 and thereafter updates over the internet are mandatory for any non-tweaked Windows, do be aware that any court order to “infect with backdoor for a particular target system + a gag order to talk about it” makes it legally possible to negate anything security wise on your system. Also, within western block supply chains, the BIOS itself (even if stored on ROM not EERAMs) cannot be trusted. So while it may simulate booting Linux, it could secretly keylog you and send it over WiFi.
- In this context we consider using “side channel-secure” libraries (eg. for modular inverse) is a performance degrading useless bandaid.

#### F. Getting randomness in practice

Here is an important note on getting “truly random nonces” and generally randomness in practice. While outside the scope of the current paper, for completeness of practicability argument, we present some ideas here also.

The following method is recommended:

1. **Start with an initial random pool consisting of the public key hashes for all addresses of any Bitcoin address which ever sent BTC on or before January 1st, 2020. This should be like 2-4 GB.**
2. Use a standard Radio/TV-Tuner to individually get some FM noise in some bandwidth and frequency which is generally understood not to be used. Note that alone this method DOES NOT generate secret randomness.
3. Use VeraCrypt about 5 times to FORMAT a 2-4 GB file after you moved the mouse like at least 2 minutes to create randomness each time. Remember to also use your left hand (or right if left-handed or ambidextral) to move the mouse also.
4. Create a 2-4 GB file which is the XOR of all the sources of entropy 1-3 above. If used in a public-replicable context, you can just cache the VeraCrypt Master Keys used for formatting and then you can skip point 2.. or just use a short sequence and publish it.
5. Starting with a 2-4 GB stuff of truly random data, each time you need like another 2-4 GB of data do the following:
  - a. Choose a secret 16 KB *sk0* private SKREMA<sub>alpha</sub> key. It would ideally be a KDF-processed version of whatever you



type XORed with a VeraCrypt MasterKey derived via mouse-movement.

- b. Generate a new VeraCrypt formatted volume with a different MasterKey (so move the mouse again to format a different volume).
- c. XOR the contents of points 4 and 5b above as a baseline B. If you need reproducibility, just publish the MasterKey in b and the sk0 in a as the public randomness key.
- d. **Decrypt B using key sk0 repeatedly (you get 16MB next block SKS with each decrypt), using as a baseline the transformed B “leftover” from the prior decrypt and the new block key.** You can do this to get any number of 16 MB blocks you need. For example, to get a fresh 4 GB of randomness sufficient for a 16MB plain text actual encryption you need 256 decryptions. So that is envisioned to require like 7 to 32 eight-hour work days – the initial setup of an “individual SKREM nonce baseline”.
- e. Remember that the above needs to be done while in BUNKER MODE. And only the result of the actual encryption over it be published (and ofc, not just DELTAs to what is generated).

#### G. Usage applications

Note that while initial setup may be time-consuming and also encrypt/decrypt of a single block is quite timely at a target 20-60 minutes, SKREMAAlpha is, for those willing and able to take the “setup time” (circa 50-256 compute-hours) and the initial “load/channel open time” (circa 20-60 minutes), suitable and designed for almost any cryptographic application such as:

1. **Instant Messaging (text and emoticons).** Just decrypt a XOR mask first (up to 16MB at a time) as “opening channel” setup. Then subsequent 16 MB as needed. Use separate blobs for sending files, sending just their SKREMAAlphaHash as per point 11 below over the IM protocol.
2. **Peer-to-peer secure file transfer.** Since encryption / decryption up to step 18 does NOT depend on plain text, Transition Maps T as per point 10 below can be computed in advance in BUNKER mode and just moved into a partly untrusted environment outside it upfront.
3. **Authentication and Authorization based on shared secret over public channel.** As per protocol described in [2].
4. **Private Lamport-like digital signatures (digest auth).** As described in [2].
5. **Public Cloud secure file archival storage / retrieval.** This means uploading / downloading / moving around data from an untrusted public cloud – which learns nothing about the content it handles. For short stuff, the public cloud can be a Web30 provider or even a blockchain. The author has envisioned a fully realizable,

efficient dataflow which requires only a BUNKER and a number of only 2 semi-trusted machines (which cannot be infected just by reading a microSD card). The full flow is out of scope for this paper.

6. **Censorship Free Content Distribution over Public, untrusted Cloud Providers.** Using the trick in point 10 below, one can upload a “encrypted original version” of some content C. Then periodically download that and transmute it locally to another <secret\_key/nonce> pair and reupload it as C2. The server cannot correlate C2 with C in any manner. Then when a customer requests the underlying content of C2 he can provide a session client secret key ck2 which locally (outside the cloud) is used to generate a transmutation  $C2 \rightarrow C3$  and a link provided to the client. Alternatively, a series of hashes as per point 11 below <hk1, d1>, <hk2, d2>, etc. can be computed over C2. Then a requesting client in a torrent-like peer-to-peer network can ask for someone having a ciphertext having hash <hk\_x, d\_x> and requesting it be transmuted to key ck2 directly to the Cloud Provider. If the service is then requested to remove C2 due to censorship reasons, the content distributor will just refresh his non-Mallory clients to a new variant C3 with a different set of hashes. For optional further obfuscation, techniques presented in [1] allow for encrypting multiple plaintexts in the same ciphertext output (so the same ciphertext could resolve to 2-3 distinct contents, thus being unclear which one the requesting user sought).
7. **Blockchain Proof-of-Work, Proof-of-Ownership, joint secure coin-flipping, etc.,** based on the other points presented. The incarnation of each of these is out of scope for this paper, but they are realizable.
8. **Any application which requires a KPRF (Keyed Probabilistic Random Function)** providing large output based on relatively short secret key and publicly known but immutable, truly random data of at least 2-4 GB size.
9. **Many other classical applications** where symmetric key ciphers are typically used.

Note that by its design, SKREMAAlpha unlike most ciphers, supports the following cool operation:

10. **SKREMAAlphaTransmuter.** It is possible to change the private secret key used for encryption securely from k1 to k2 remotely. So basically instructing an untrusted Cloud Service for example to do a transformation T over the encrypted data E which has the effect of transforming E into F, with F encrypting the same plaintext as E, but having the secret key k2 instead of k1. The transformation T is information theoretically secure and reveals nothing about neither k1, k2 nor the plain-text. In fact, one can move from k1 to a number of keys k2, k3, k4 etc. without compromising security. Also moving from k1->k2->k3->.. also is possible without compromising security. In order to compute T, one requires access to both k1, k2 and either E or the original GMT nonce [NB: plaintext knowledge is NOT required]. So computing Tk1->k2 transformation data to

be later operated by an untrusted cloud service is possible for an original encryptor of data.

Using the cool operation 10 above and/or SKREMA $\alpha$  in KPRF mode, the following use case are also supported:

11. **SKREMA $\alpha$ Hash.** Secure, quality hashing of large volumes of data. This is done like this: Consider an initial secret key  $k_1$  and a fixed GMT  $M$ , public but not writeable by any adversary (potentially being setup using the random coinflip protocol introduced in [2]). Using technique 5 above generate as much randomness as needed from  $k_1$  and  $M$ . Then repeatedly decrypt, each time XORing the SKS for the next block with the next 16 MB of the plain text, in sequence. Finally one gets 16 MB – the final effective key. Do yet a final round of decryption with this effective key and select as much bits from the decryption result as needed – so any digest  $d$  from 1 bit up to 16MB is possible. The hash is  $\langle k_1, d \rangle$ . Note that this hashing scheme is claimed to be secure and is preferred to the one introduced in [2]. The latter had an incomplete description and required very careful setup which was not discussed to avoid being trivially insecure.
12. **Authenticated symmetric-key Encryption.** AEAD is a common theme lately in cryptography. Most schemes the user consulted from OSINT papers are a bit hair raising to him – they seem to offer the opportunity for CyberMallory time attackers to get some guaranteed correlations to work with, regardless even of plain text. Using SKREMA $\alpha$  one can do AEAD simply like this: Reduce number of 2-bit pairs emitted bits per plain-text bit in Step 18 from 2 to 1. Every other emitted bit thus becomes unused. Compute the SKREMA $\alpha$ Hash as per point 11 above of the entire cryptotext block, however considering a fixed nonce  $n_B$  instead of the newly ignored emitted bits (so the corresponding 2-bit locations are taken to be replaced by the nonce  $n_B$  or simply be considered to all be 00/01/10/11 alternatingly). This gives a hash  $\langle hk, d \rangle$ . Now complete the encryption in Step 18 to encode  $\langle hk, d \rangle$  in the priorly newly ignored bits. Note that this schemes computes a hash over both plain-text-encoding-locations and their values, and also over the randomness providing locations (2GB – 16MB of them, so ~99.2%). To authenticate an incoming message just repeat the above and compare obtained digest to the values encoded in the free bits. To circumvent the scheme, an attacker would have to know the secret key used initially.
13. **FHE-like Data-secure computations using a Public Cloud.** Now this is a good one. Here's the model: Alice (the cloud provider) has a cool algorithm  $A$  which can be encoded as a Turing Machine  $T$  running in maximal time  $t$  steps and using maximally  $q$  tape bits. Bob has some secret data  $P$  he wishes to be processed using the algorithm  $A$  to get the end result  $R$ . Now Alice wants to keep her algorithm secret, without revealing to Bob anything beyond the result its operations on the data  $P$  plus some constant number of random data per run. Bob wants to keep both his input data  $P$  and the result of the

processing  $R$  secret from Alice. Note that in the more practical setting, Bob actually uploads some algorithm to a Public Cloud provider (or a Web30 mash) which he then wants to run on his data using the remote compute power, not that of his laptop. This is achieved as follows: a) Alice and Bob do a secure coinflip / commit somehow to a vector of  $O(|P|)$  truly random GMTs. Say  $10 \cdot |P|$ . The first  $|P|$  are used to represent the input, the next  $|P|$  to represent logical 1 and the subsequent  $|P|$  to represent logical zero and the rest random data. Bob keeps as his private secret key some index  $idx_i$  for each element of the vector, telling him “which bit index from the GMT is the plain text bit”. Then b) Bob sends Alice a number of Transmutations  $T_{ij}$ , with  $i=0..t$  and  $j=0..q$  so that  $T_{ik}$  and  $T_{jk}$  map the “plain text bit” (if any) of bit register  $i$  of layer  $l$  and that of register  $j$  of layer  $l$ , into the same “plain text bit” of register  $k$  at layer  $l+1$ . The other bit indexes are permuted randomly. This way Bob can keep track, for each layer of simulating  $T$  as a Boolean circuit, the location of the “plain text” bits of any tape-mapped variable register. Then c) Alice runs  $A$  by using the Boolean Circuit representation of  $T$ , and using GMT-sized variable registers and doing point-bitwise operations on all indexes in them when a logical operation is required. Finally, d) When done, Alice sends only the final layer  $l$  tape registers corresponding to the output bits of algorithm  $A$  to Bob and tells him the indexes of each in the final layer. Then e) Using his secret key, Bob can track starting from layer 0 up to the final layer  $t$ , the location of the “plain text” bit for each index location. This way, based on the data he received from Alice, he gets just the output of running  $A$  on  $P$ , plus a number of  $|GMT|-1$  outputs of running  $A$  on junk random data (supposedly uninformative for Bob). This way Alice did not reveal her algorithm, Bob did not reveal his input data, nor his output data to Alice. **This is a pretty cool stuff!** Note that it is semantically similar, but fully distinct from both functional encryption (in that setup Alice sends a modified version of  $A$  to Bob, in this variant, Bob sends a modified version of  $P$  to Alice) and Fully Homomorphic Encryption (FHE) (in that setup the encrypted data itself can be acted upon freely without special steps to initialize / decrypt or special setup for operations: simply encrypt, do additions, subtractions and whatever else supported by the scheme, and then decrypt).

If the materialization of the above is interesting to the reader but details remain unclear, we also recommend our prior paper “Applications of SKREM-like ciphers” [2]. Alternatively or complementary, the reader is also encouraged to contact the author directly for consultancy. Contact details are linked in Section 9.

## V. METHODS AND EDUCATED PREDICTIONS

The timeframe 2021-2026, since the initial paper on SKREM-like ciphers was published and today, and moreso the timeframe 2019-2026 when the results in the 2021 paper

were initially crystalized in unpublished form, has seen major evolution in attack vectors and was also a fruitful period of discovery by the author of both ample raw data based NEWSINT and OSINT and also of finished ideas, produced by critical judgement, theoretical and practical modeling (mainly using ChatGPT v4.0 simulations and chats) and, of course, of processing and operationalizing the ample raw data which favorable fortune endowed the author to collect.

The most relevant fountain of information is a HUMINT encounter in 2022 in Heidelberg, Germany, when the author had a change to exfiltrate a bit a very high-ranking person of the western block establishment from competent Western (USA) institutions with direct and ample responsibilities for both offensive and defensive cryptography. The details of that encounter are outside the scope of this strictly technical paper – and also in part as (a then-unreciprocated!) professional curtsy to the person in question will not be published nor discussed here. Nevertheless, some of the major conclusions drawn and hypothesis formulated as a result of that November encounter together with all the other sources of raw data (OSINT, NEWSINT, other HUMINT) – which for brevity and partly OOO issues will also not be published here in raw form –, combined with the critical thinking and theoretical and practical modelling, ARE presented here in terms of end-result production: hypothesis, claims and predictions.

Additionally in terms of HUMINT is the author's personal experience with the military of an important country, where they demonstrated the capability to compromised translate.google.com so that, while using his own phone, his own browser showing the connection as Secure (by SSL and not TLS1.0!), having his own name in English "Mircea Digulescu" translate to the foreign language equivalent of "sexy videos" and other such nice teasers.

Additionally in terms of SIGINT, Mircea Digulescu's own IT assets (laptop, PCs) where, in 2021 compromised in the most funny and peculiar ways (see [x]), a success attributed in part also to breaking cryptographic signature keys of some software vendors (or the PRNGs used to generate them based on their seeds).

In terms of NEWSINT we rely mainly on repeated failures of military attack drones, surveillance drones, and generally the compromise of adversary's IT systems both fixed and mobile during the timeframe 2022-2026 in all combat theaters: Russia-Ukraine, Iran, Lebanon and Israel/Palestine, Venezuela and space. Stuff such as hijacking of adversary's drones (by Ukraine against Russia, by Iran against USA), by compromise of military C4 systems (by Ukraine against Russia and Russia against Ukraine – several known instances), by conjectured compromise of non-backdoored IT systems of Hezbollah by Israel (leading to termination of a sequence of Hezbollah leaders) and others.

In terms of OSINT, we quote the successful AI/ML based attacks against reduced-round AES and against SPECK, all done on modest compute resources, with modest precompute times.

In terms of critical judgement and theoretical modeling, we quote like 2-3 years of additional consideration research wise of the problem of cryptography very intensely so post the 2022 encounter.

In terms of modeling and practical simulations, as well as preliminary (both vetted and verified and taken as unverified but likely true), we quote months of almost daily, hours long chats with ChatGPT v4.0 oftentimes resulting in C# or other code which was ran as simulations by the system: especially to test variants of Random-G and of ModInvCSPRNG but also to do literary review (some double checked for factuality) and "stress-test" our constructs from the point of view of OSINT public cryptography "talk".

Additionally, SKREM-like ciphers and some later variant formed the topic of intense (and sometimes passionate) talks and replies on the r/crypto Reddit sub, before, post-2023, the author was getting his account repeatedly banned and then reinstated (on appeal) months later by Reddit.com when trying to post again to r/crypto.

While compiling the raw data for all of the above is a feasible task, we consider it out of scope in the context of this paper – not even as an annex, not even on demand. If the reader cares about some punctual questions / challenges, he or she is encouraged to reach out to the author directly or for consultancy services. Links to contact data are available in the Vitae Section 8. Nevertheless, simply the above notes serve to illustrate that even absent formal proofs or deep arguments, the author is both savvy in the field, has had access to ample and relevant raw data, and has used sound methods and practices and also reasonable and significant effort to derive the results and claims of this paper.

So, based on everything 2021-2026, here are some great statements (claims, hypothesis, conjectures, feelings, whatever) which are presented as is, which we the author hold with high or very high confidence level, and which, in the context of this paper or not elaborated on further:

1. All current NIST standards ciphers and cryptographic primitives are largely breakable in 10-20 minutes top by a super-power state actor adversary. By "largely breakable" we mean that at the very minimum, given an output a preimage can be produced efficiently. Some of the ciphers thus claimed breakable are: AES (Rijndael), all other NIST AES candidates for symmetric key (Serpent, etc.), Keccak-f, SHA2, Feistel networks with many iterations, and many others, except RSA (which can be broken shortly enough with quantum computers). The attack scenario is "known plain text" / known output of small but sufficient size.
2. All current "short, fast, small and simple" ciphers known to the cryptographic community (with the potential exception of some strong variants of McEliece about which I am unsure), as well as SKREM in the original variant are also breakable in the same conditions as point 1 above. By short, I mean any cipher were "the core security" lies in a short description ["short C# program size entropy-wise"], fixed (uniform) algorithm (including in its size fixed S-Boxes, libraries used, etc.). By "fast" I mean the number of steps performed on the CPU/RAM to produce an output block

– anything which is constant size and linear in like 10,000 is considered fast. By small I mean the core algorithm transforms a small block input (meaning like less than 1MB) into another small block output (usually the same size, but always still less than like 1-10MB). By simple, I mean the core security relies on combining a small number of local and not very complicated (generally  $O(1)$  depth) transformations, even if repeated over a number of rounds – so Keccak-f Chi and all the others qualify – and additionally anything which ultimately induces a “weak SAT circuit” [a sort of uninformative definition, we admit]. Additionally, the attack vector was credibly claimed to break a “long, slow, large but simple” cipher like the initial variant of SKREM (which had a number of beginner issues to begin with).

3. Any arbitrary “*short, fast, small and simple*” cipher, including prospective and future ones, any “*sufficiently simple cipher*” and, in the near future, if not already, any “*short, fast and small*” cipher even if not simple, is breakable by the current attack vectors in the same conditions as point 1, potentially with a few hours-to-days of preprocessing required at first (for an NSA-computing/storage power like actor).
4. Some surprising non-trivial cipher ideas are also assessed by us to be breakable in the same conditions as point 1 above (the weakness of some being an open secret): ciphers based on NP-complete problem instances with trapdoors [including properly fully randomly built 3CNF-SAT instances with a single forced satisfying assignment – of reasonable size (up to 1000 to potentially 10,000 or 1 million if construction is weak or faulty)].
5. The most likely setup of the most effective attack vector (and the one most likely to be used by the top western block qualified agencies) for breaking the all of the above is as follows. An AI/ML model is trained on the following data: (input secret key, plain text)  $\rightarrow$  (output) pairs. Additionally, most likely, the intermediate memory evolution of the cipher-under-attack (SUT) is also fed as input to the model. If a common occurring plain text (eg. “Windows”, “MZ”, “this program cannot be ran in dos mode”, “attack at dawn”, “withdraw to the mountains”, “I need more cheese”, “sauna is too hot”, etc.) is suspected for actual practical targets, that is used directly instead of arbitrary plain-text pairs. The AI/ML model then develops final and intermediate “correlations”. A correlation is something like: “If the secret key obeys property PS1 then the output obeys property OP1”. So PS1 can range from like “the first 5 bits are 11001” to “the hamming distance of this key versus the baseline is  $<5$ ” and basically anything – so its not from a preset set: most likely the AI/ML model “learns” to build up correlations’ statements, or, if the cipher is short, just Boolean formulas over (part of) the input. In terms of  $O(1)$  it can again be anything (typically short but like can be up to something like 10-100KB in encoding). So by being trained on a data set of (secret key, plain text)  $\rightarrow$  internal memory state1  $\rightarrow$  internal

memory state 2  $\rightarrow$  ...  $\rightarrow$  (output), the AI/ML attacking model “learns” invariants of the SUT. Note that the training is probably done adaptively: so not just arbitrary pairs, but it’s like an interactive Codeforces.com problem – prior learning results are used to constrain (part of) future queries. Also, it is probably not simple (input-output) pairs but also intermediate memory states of the SUT which are fed in: although intermediate memory states are not public in real attacks, they do form a good intermediate point to which to bind correlations. Also, almost surely the model incorporates “self-learning” – it produces components which “learn to learn certain aspects of the process” – producing an inner agent of sorts. Finally, it is very likely the attacking AI/ML model is “self-reflective” – namely it learns from its prior success and failures to more aptly make the open choices and formulate future queries. Beyond finally, it is most likely that the model will soon also be agentic and non-uniform (hey!, exfiltration between to competent parties is never one way.. it goes both ways). In the context of AES – as well as of other ciphers for sure – we suspect also that subject-matter expertise was used initially to fine-tune the learning and make it converge faster. Also, we expect that this attacking AI/ML model (lets call it CyberMallory) was trained and perfected since circa 20 years ago. Also, in case of AES, it probably had like 10-15 years+ of preprocessing (learning general correlations and learning to learn correlations for a fixed known plain text) on data-center level compute power, probably producing on the order of Peta-to-Exabytes of precomputed knowledge. The end result? We project CyberMallory is a private-cloud service which is fed something like (i) a C# program say McBob [the encryption and decryption algorithm], (ii) known input-output correlations, (iii) some bounds on resources to be used (iv) some timeframe for preprocessing. Once ran, it produces another private-cloud service called CyberMalloryForMcBob which, when fed a known plain text and a cipher text attempts to break it, similarly to points (ii) and (iii) above. Now for the classes of ciphers mentioned in points (1)-(4), I expect that in like a few hours of running CyberMallory, the CyberMalloryForMcBob can with probability of like  $1/e$  (63%) break at least 50% of plaintext-cipher text pairs (especially when plaintext common like OS constant strings, file format constants or checksums, etc.) when ran for like 20-60 minutes. So this includes AES – where learning to learn to break ciphers was probably useful for rollout to “general C# program case”. So while the algorithm is not even explicitly converted to a SAT circuit/formula, it is nevertheless breakable automatically in most cases.

6. What do we asses to be secure in the above context? Well, in the long term - very difficult to say. However, we claim that at least one of following is true: (A) SKREMAAlpha or a very similar cipher is secure, OR (B) nothing is secure (ie  $P \approx NP$  in practice) OR (C) the shortest secure cipher does not admit a brief description

(ie. probably the shortest secure-ish cipher C# program – with all dependencies and excluding any random bits sampled at runtime – is beyond 10-100MB large in *entropy size*) OR (D) the relationship (cipher – successful attacker) does not admit any supreme point (ie. no fixed cipher unbreakable by all attackers up to relevant size) but instead it has a relatively large Pareto frontier -- this is what we the author actually suspect of solving NP-Complete on the finite case (actually with a much smaller Pareto frontier there) OR (E) the shortest secure cipher exists but is highly irregular and hard to find by chance even with AI/ML assistance (although the latter helps) OR (F) the shortest secure cipher actually has a very long running time [as it probably enumerates entropy until some unintuitive and fixed point where it simulates another program or such] OR (G) the shortest secure cipher makes use of some Information Theoretical non-Turing aspects of mathematical truths (as hinted to in section 7). A combination of these is unlikely, however (G) and (F) can ofc coexist with all the others. So what do we expect in practice? We conjecture that for the moment (up until towards the year 2099) that (A) holds true. Also, we expect that (B) is true almost all short, non-uniform, unhinted algorithms - - yet some of them would remain secure for many decades still. We expect (C), (D) and (G) to produce very interesting ciphers which, without deciding the other statements will coexist with SKREMAAlpha -like and AI/ML generated ciphers for the rest of the 21<sup>st</sup> century. Ultimately, going into 2100 and beyond, we expect that (1) the classical computer model will still be relevant and that (2) only ciphers with long running times (like millions to billions to trillions of steps) combining ideas from (A), (C) and (G) with AI/ML assisted improvements resulting in non-uniform, incremental agentic families of long [program source code + hint + dependencies] ciphers. Statement (D) will probably be true in an information theoretical sense, but without a practical application until mid-22<sup>nd</sup> century. The above will hold even in face of Moore's law over the 21<sup>st</sup> century. We also expect that ciphers so produced will not be attacked successfully by computers based on new physical principles, quantum computers and generally except for speedups not by new computational models.

7. **For now and at least the next few decades (up until 2050 at least), we claim SKREMAAlpha to be secure. Including for use as KPRF.**
8. Only thorough testing of SKREMAAlpha (preferably against CyberMallory itself) can determine if SKREMAAlpha (or anything which any cryptography researcher could think up himself and put in words in a 100-1000 pages or less paper) is actually secure or not. The author will attempt to gather some additional intelligence against of performance of SKREMAAlpha (or some trimmed down version of it at least) against some systems like CyberMallory – it is unknown and out of scope if he will be successful in this or whether he will live to publish the findings.

9. A properly financed project to research security of SKREMAAlpha and such ciphers is truly required in the modern attack vector era. Some directions into what and how to test are presented in Section 7.
10. We the author project that running time will be a critical success factor for future secure ciphers (in the sense that longer is better). It is possible that in a very short while, only state-like or select non-state actors with technological edge and vast compute power will be able to field and run secure ciphers: both in terms of computer-aided design of them (AI/ML) and in terms of actually fitting a secure enough running time complexity into a short enough practical time – so photonic computing, dedicated hardware, better phototransistor plates, etc.. A key component of future secure cipher will be a rather long encryption/operation running time. So if SKREMAAlpha for example takes like 1 hour to encrypt 16 MB, then a more resourceful and higher-tech actor, could fit some Section 7 enhancement plus multiple rounds of SKREMAAlpha or other stuff in like 5 minutes – while an average Joe, on his PC or laptop in BUNKER MODE would take years or millennia to run the same, and/or run out of space. So cloud-based encryption/decryption services (if marketed or allowed for such purposes officially) are expected to become “a thing” in like 5-10 years.
11. Secret split (XOR) or k-of-n secret sharing over multiple jurisdictions (Canada, SUA, Brazil, Peru, Venezuela, Uganda, Kenya, Niger, Egypt, Israel, Iran, Armenia, Georgia, Russia, China, North Korea, South Korea, Vietnam, Nepal, Bhutan, Japan, Australia, New Zealand, Italy, France, Germany, Estonia, Finland, Denmark, Belarus) in perhaps like k-of-(l-of-m)-of-n [grouping some jurisdictions together in say l=5 political blocks], with like 3-of-5 secret sharing at that highest level and like 3-of-7 within the block, combined with a decent but not fully secure cipher, coupled with large data volumes (so the Telegram model) is likely to offer quite a lot in practical terms for most of what is understood as non-military information security than ciphers. Even with good ciphers like SKREMAAlpha, cross-data center secret split (perhaps even via mediation of a non-airgapped but non-backdoored secure personal laptop) is and will be quite “a thing”.
12. Data stored on LEO satellites will become readable remotely to high-tech state actors via ground-based systems in the next 5-10 years (if not already). It will become corruptible in like 10-15 (if not already). It might also become writable in part or full by around 2050 (think discombulator-like devices enhanced with AI/ML targeting and lots of trial and error). So good ciphers will be yet again even more relevant to state actors, and to private companies operating satellites.
13. We project that, of all high-tech actors, USA is the most advanced. In practical terms we expect that, beyond points 1-4 above, relevant agencies will be able to break the current cipher primitives on the Bitcoin blockchain. As of 2022 we project the were already able to invert a

public key has into a bitstream input, however where not able to further constrain that input to be an RSA prime so as to be able to sign a transaction with it. We project that in the next 5 years (if not already) they will have overcome that limitations.

14. We project that point 13 above even when it becomes public or an open secret, will not mean the end of Bitcoin. Not at all. Instead, at some future point within the next 5-10 years, the miners on the BTC block chain will agree to switch to new cryptographic primitives (maybe ones based on SKREMA $\alpha$ , why not :) and also revert the last like 2-4 then-weeks of transactions. However, we expect that quite a large number of BTC addresses will genuinely be broken by both state-actors and by hackers and by criminal groups (in all overlaps and disjointness) like 1-2 years before the switch is made. No doubt some more will simply be claimed to having been broken for various privacy reasons. However overall we expect that in 5-10 years BTC will operate yet some more based on new cryptographic primitives. However, the confidence and performance of the new primitives will no longer be “the comfortable roll in the park” of what the early 21st century offered in terms of practical cryptography.
15. Similarly, we expect all block chains (especially ones we perceive as semi-centralized like ETH, Ripple, etc.) perhaps with the exception of Monero (due to Random-X, but they still need to reform other crypto primitives) will suffer the same fate and transformations as per points 13-14 above.
16. We conjecture that in  $P \approx NP$  on the finite case, in the following sense: **“There exists a small sized non-uniform, learnable family of hinted algorithms  $A[n]_h[n]$ , which has no more than  $O(n)$  members and each member is no more than  $O(n^2 \cdot n^4)$  in size, including hints, which  $A[n]_h[n](x)$  can in finite complexity  $Poly(n)$  or better, solve at least 50% of all SAT instances  $x$  of less than  $n$  variables and description size less than  $n$ , with probability greater than 63%”**. So basically, we claim that NP is in BPP/poly and that furthermore it can be learned, at the very minimum up to like 1000-1M variables, *except* some very small number of families of cases which themselves will be very hard to find or characterize.
17. We project (without conjecturing) that NP is most likely in learnable BPP/poly in the general case as well (with a rather long preprocessing time), **except some families of cases – some of them succinctly describable others not**. So, we project than in 20-25 years, “The fundamental theorem of computing” will solve like about logarithmic-95%  $x$  of instances of NP-complete problems up to some large finite limit (beyond 1 million) on several finite and several enumerable categories of instances. Thus, it will mimic on a much more sophisticated scale the size and breadth and importance of The Fundamental Theory of Algebra. Unlike the latter, the former will remain incomplete beyond 2050 and beyond the 21<sup>st</sup> century.

18. We project that in the general uniform case (so without preprocessing time as a function of  $n$  but at most constant) that NP is outside P but below EXP.

19. We conjecture that in the general BUT non-uniform case, that  $P=NP$  in terms of finite case complexity (NP[n] belongs to Poly[n] and it never explodes beyond it). Namely conjecture there exists a non-uniform (and we believe also learnable except with a few exceptions here and there) family  $A[]$  of total size Poly( $n$ ) which offers a running time of Poly( $n$ ) for solving NP SAT instance up to arbitrary large  $n$ . Building the family  $A[]$  might take beyond Poly( $n$ ) time though – thus keeping the uniform general case outside P (so still  $NP>P$  generally, but on any arbitrary large finite case NP[n] in Poly[n]). So paying a one-time large cost of say  $2^{\sqrt[4]{10^{106}}}$  compute steps (maybe via future accelerated Turing Machines) can provide a succinct  $A[]$  solver of size SemiPoly[n] ( $\sim n^{\log n}$ ) which solves any instance of size up to  $n$ , in Poly[n] time ( $\sim n^{\log \log n}$ ). We also project there exists many (and even classes of) instances of NP-Complete problems which do NOT have polynomial sized certificates of non-solvability. For these, only the soundness of the construction and execution of the “build-up algorithm” of the  $A[]$  family will serve as proof that when  $A[n](x)$  outputs “unsolvable but without short proof” on some input  $x$  that the statement holds true.

20. We expect that despite all the points above, even during the timeframe 2050-2099 and thereafter (with new cryptographic primitives) still there will exist many ciphers about which it is hard to reason in practice if they are breakable or not, and even for state-size then-hightech-actors impossible or hard to break in practice. Up until 2050 we expect SKREMA $\alpha$  at most with some small improvements / enhancements and such ciphers to remain secure. For 2050-2099 we expect that some “quality” random (yet secretly built) nonces (like with information theoretical backdoors incorporated for some future, currently not formulated problems – think in terms of Busy Beaver) will start to be incorporated into ciphers in order for them to be considerable for being considered secure by the then-standards. Building such nonces will probably be computer-aided design and (even for a single individual nonce) will probably require human input, up until at least the year 2099.

21. So, in a nutshell, we conclude with this statement: **AES (Rijndael-like) ciphers were definitely the ciphers of the early 21st century, however for sure they will not be the cipher of the end of the 21st century – in fact not even of the mid-21st century. SKREMA $\alpha$  and similar but unenhanced ciphers, coexisting with some computer-aided designed ones, will be the ciphers of the mid-21st century.** During the late 21st century the spectrum of computing and cryptography will change unrecognizably for some from what it is today. **Nevertheless, symmetric key ciphers build in a computer-aided fashion, based on the classical Turing-machine model and the techniques and strategies presented in this paper, which will**

**start to spur close the turn of the 21st century, will continue to be secure in practice against all then-modern realizable computational models in practice, well into the 22nd century.**

22. In terms of Objective Reality physics, We the author do not know if Cryptography is genuinely possible as an Information Theoretical problem, against what is physically realizable in full effect as a Computing Model. We do know that Information Security is NOT – so no “private, unseeable” hardware – which itself makes practical cryptography useless against such “Unity Device”-like actors. Ultimately, if we were to take a guess, we would say that both Yes, and then No and the Yes and so on. Meaning that, into the far future, simply defining something like a secure cipher under any then-feasible computational model (even a theoretical one) will give rise to a (potentially new) physically realizable computation model which breaks it. And then, in the new context of computational models, yet another secure cipher can be designed, which yet again is breakable by expanding the computational models’ space. Sort of like a Hyperphysical Hierarchy of Computational Models.

23. The current best computational model we have is the Turing Machine. It will remain to be relevant way into the 22nd century. However, may others beyond it exist – especially in terms of calculability and complexity on the non-uniform, finite case. Some compute models which are currently known to be physically materializable, at least theoretically, just throwing some arrows here are: *photosynthesis based quantum path finding* (better complexity classes), *Blum-Shub-Smale seeded with arbitrary reals or reals sampled from physical reality* of the current Q-Universe (current Cosmos like with stars and galaxies), *accelerated Turing Machines* (materialized by moving the observer close to the event horizon of a black hole), *Baryon-Qualion* computers (Turing Machines or other systems which import acts and thought patterns produced by living beings as an Oracle – eg. a computer which acts upon a cat or ameoboma classically and then imports some aspect of its reactions or thought patterns back internally). Many more powerful, physically realizable models we conjecture to exist in terms of the Absoluteness of the existence – perhaps even uncountably infinitely many qualitatively distinct ones, forming an uncountable infinite Pareto frontier even.

## VI. PUTTING SKREMA ALPHA TO THE TEST

We dedicate this section to an overview of minimalistic practical experiments which should be carried out on SKREMA Alpha (or any cipher for that matter) as part of its signoff. This is not a blueprint for ruling SKREMA Alpha secure, just for ruling out it being trivially flawed.

Here is what should be tested. Firstly, a trimmed down SKREMA Alpha variant, SKREMA Alpha Blue (SAB) should be contrived having:

a) smaller size of prime numbers used for CSCSPRNG (like 32-64 bits).

- b) smaller size of GMT – maybe down to 16 MB.
- c) smaller number of Turing steps – perhaps down to simulating 0.5-1 MB.
- d) smaller sized effective key (32-64 bits).

Now, independently both SKREMA Alpha and SAB should be tested for quality both globally (input – output pairs) and **each step (0-20) independently**. Quality (security testing) involves ascertaining the following:

- a) quality of randomness produced, given input
- b) lack of usable input-output, or input-intermediate / intermediate – output correlations produced.

Practically, for step a) we recommend the following:

1. Start with random inputs  $k_1, k_2, \dots, k_n$ .
2. For each input, ascertain via *randomness assessors* the quality of randomness produced. One can consider the DIEHARD tests here, perhaps after slight adjustment for what is actually produced in the respective step.
3. For each input, modify the first, the middle, the last, then 5 other random single bits. Additionally do 8 more random flips of 1 single byte. Then choose 16-32 bits at random and flip them, for 8 more samples. Then flip the entire input randomly for 8 more samples ( $8 \times 4 = 32$  total). Assess randomness of the outputs treated independently. Then Assess how many bits of output change (ideally 50%). Test the pattern of bit change correlations considering: entropy, average hamming distance between output samples, symbolic correlations.
- For SAB, do symbolic correlations attacks: fix all but 16-32 bits of input, and then compute the algorithm symbolic, obtaining SAT expressions for each output bit. Select  $8 \times 8$  such output expressions at random and feed them into a SAT Solver.
- For SAB, do CyberMallory-type attacks, by feeding (input, output) pairs -- even (input, memory history, output) tuples if possible -- to SMT or an AI/ML correlation attacker model. Check to see how long it takes for it to recover the key. How many bits needed to be fixed, etc.

Note that the ultimate output of SKREMA Alpha – produced in Step 18, is a *partial permutation*. There are many other steps, and also the Digulescian Permuter, all of which produce permutations as output. Since there are currently no DIEHARD tests for permutations, the following transformations to what is actually fed into DIEHARD can be made:

- Consider consecutive pairs in the partial permutation produced. Map each of Odd/Even, Low/High, left < right to bits, and perform the above tests on these bits.
- Check if the number of inversions, both for the overall output and for 50% of it selected randomly 8 times is within 2-3 sigma of its expected theoretical value for a truly random permutation.

- To check quality of selection, consider the distance between consecutive selected indexes. In fact, consider the difference of that from its expected average. Then do the above tests.
- Finally, simply apply the selection permutation to 010101.. and do the above tests.

Additionally, both for SKREMAAlpha and SAB, baseline canonical vectors need to be established and published (input-output correlations).

Note the just the tasks put forth in this section 6, can be understood to require some noticeable amount of engineering effort, which is typically understood to be done in the context of a financed project.

## VII. MODELING SECURITY OF CRYPTOGRAPHIC SYSTEMS AGAINST MODERN ATTACK VECTORS

We dedicate this very brief section to a formula we use internally to check the expected practical security of symmetric key cipher.

Let's say the cipher is claimed / proven to offer security of **k bits** against a specific type of attack (eg. brute-force).

Now in evaluating what is the effective security for a more powerful adversary, over many years, we resort to the following formula.

Consider:

- a = degradation factor due to better generic AI/ML CyberMallory learning model/algorithms. Typically taken to be 2-8.
- b = degradation factor due to preprocessing over 20 years. Typically taken to be 2-6.
- c = degradation factor due to better incorporation of subject-matter expertise (eg. better deep finite maths knowledge, better prime field theory results, etc.). Typically taken to be 2-3.
- d = degradation factor due to larger memory for meet-in-the-middle or memory/time tradeoffs. Typically taken to be 2-3.
- E = degradation term due to acceptable probability of failure for attacker. Understood to be the log2 of that probability, which is set at  $2^{10} - 2^{20}$ , so e is taken to be ~33-67.
- A = CPU-count/FLOPS advantage term for datacenter attacker versus desktop/laptop. Note that for SKREMAAlpha, if brute-force attacks are done or similar, actually RAM limits may be determinant (~2-4 GB required). Understood typically to be the log2 of the FLOPS of the datacenter, which was around 1 EXAFLOP for NSA datacenter circa 2020, so  $10^{18}$ , so A ~ 60.
- B = the advantage term for the attacker using better hardware (eg. attosecond photonic computing versus classical). Understood to be the log2 of the multiply factor for speed (around  $10^9$  for photonic computing) so B ~ 30.
- C = the advantage term for the attacker taking a lot of time (1 hour to 100 years) to try to break the

cipher. In practice 1 hour, design for 100 years. Taken as log2 of the number of seconds in the timeframe. So, we take C ~ 12 (practice) or C ~ 32 (design time).

- m = factor to account for projected Moore law improvements, favoring the attacker over the relevant timeframe. Given the current bottlenecks, we take m = 1-3.
- n = factor to account for projected breakthroughs and general improvements in approaches by the attacker, over the timeframe. Once AI/ML/SMTs are considered, we take generally take n=1-3.
- q = factor to account for usage of QTM (Quantum Turing Machine) quantum algorithms and computers (if feasibly scaled to cipher-sized circuits). In practice q~1, but we take q=2-4 at design time (to account for Groover). Note the huge threat from photosynthesis-inspired quantum computers which We the author conjecture to be able to solve most arbitrary SAT formulas, representable in the respective surface, instantly.
- p = factor to account for unexpected luck and for safety margins when designing a cipher. Taken ~1-2.

The formula below is understood to give the expected bits of extra security against the adversary in the modeled scenario.

$$e_f * p = (k)/(a*b*c*d*m*n*q) - (A+B+C+E) \quad (2)$$

Then, iff  $e_f > 1$ , the cipher is expected to be secure.

So given the rough suggested values, this looks like this:

Pessimistically,  $e_f * 2 = k / 7776 - 189$ .

Optimistically,  $e_f * 1 = k / 16 - 135$ .

So, for SKREMAAlpha, we have:

- For 16MB effective key,  $e_f = [8535 .. 2^{32}] > 1$ .
- For 1MB effective key,  $e_f = [444..2^{28}] > 1$ .
- For 16KB effective key,  $e_f = [-86..4028] < 1 / > 1$ .
- For 0.5KB effective key,  $e_f = [-94..61] < 1 / > 1$ .
- For 256-bit effective key,  $e_f = [-94..-119] << 1$ .

Note that in practice we expect the scenario to be much closer to the optimistic scenario than the pessimistic one. So given the above, we hold SKREMAAlpha even with 16 KB key as secure for the moment – but larger keys should be offered in the longer run over the current 21st century.

Also, based on the above, note that present day public domain, mainstream ciphers appear grossly insecure.

In fact, in the near future, an effective, properly used, secret key of at least around 1 MB is considered to be sufficient. Due to the construction of SKREM, the derivation of the 16MB key based on an initial 16KB effective key (which itself can, if really needed, be derived from as low as 0.5KB user provided key), is considered sufficient (and practical) for the moment, for non-state actor scenarios. In the future,



against all actors, an effective user-provided initial secret key of at least 1-16MB is recommended.

## VIII. THEORETICAL ENHANCEMENTS TO SKREMA ALPHA AND KPRF CIPHERS

### A. General Approach

The core idea behind the gradual import of entropy from the public nonce in SKREMA Alpha, combined with the KGB step, tends to the concept of denying the adversary access to (a large part of) the actual circuit eventually used to hide the plain text. That would be like the Second Turing Step coupled with the subsequent enhancement. Up until then, what SKREMA Alpha did was essentially hide the input to that step, via a construct which – in itself – is claimed to behave like a random circuit.

Given the modern attack spectrum and its prospective advancement over the remainder of the 21st century and beyond, the question becomes Can we even better in some way? Of course, one can resort to improving the SKREMA Alpha Steps in the hopes of getting a more sophisticated circuit (so moving further away from a “simple” cipher), for example by resorting to the AI/ML computer aided techniques mentioned in section 2B.

However, like SKREM-like ciphers changed the field by starting with massive amounts of entropy as a nonce and gradually importing it – thus obtaining circuits of GB entropy (or conversely secret input of billions of variables), like the first half GMT processing changed the attack model by denying the attacker “easy” access to the public nonce entropy itself, *is there any other way in which the security can be qualitatively improved?*

The answer appears to be a resounding “YES, but..”. The but pertains to the rather prohibitive performance implications – running time and cryptotext size. However, just like the initial variant of SKREM was not practical performance wise, however later giving rise to SKREMA Alpha, so can most likely, the techniques presented below be fine-tuned so as to become fully practical. Also, for actors with access to vast amounts of compute power and storage – not only state actors, but even private companies with datacenters or with secure access there-to (Web30 for example), even these added overheads can make sense at times perhaps.

As the steps on the first half of the GMT of SKREMA Alpha were meant to deny the attacker “easy” access to full the circuit / truly random input, now we present techniques to deny him easy access also to the output of SKREMA Alpha and, in maniac mode, even in part to its input. This means that even knowing the plain-text (or part-thereof) no longer entails knowing the output of SKREM, or being able to easily “test a hypothesis” working backwards from the expected output. In maniac mode, not even easily knowing the GMT.

The way we do this is by no longer simply masking the plaintext essentially with a defacto XOR with the bitstream generated by the final sequence of locations in Step 18 (in fact we incorporate the XOR by changing the values at those

locations to give the plain text), but instead using the space for plain-text to encode essentially random-looking noise which, only when recovered successfully, can give to the proper user information about the plain text.

In the prior model, an attacker with plain text knowledge could perhaps guess like a set of 28 final location pairs and be able to quickly “rule out” those which cannot fit – since they do not produce the plain text prefix he knows. In terms of correlations, again, a CyberMallory or CyberMalloryForSKREMA Alpha might potentially find some general correlations in (input, output) pairs in principle or for related keys -- in SKREMA Alpha circuit were “too simple” (which we expect to NOT be the case). However, without being able to test the materialization of those correlations in the concrete case of the (secret key, plain text) pair under attack, the adversary cannot progress into trying to gradually learn the key or additional bits of the plain text or anything about the internal operation. The way SKREMA Alpha is constructed, the output the attacker sees (the modified nonce) – is valid for any secret key – so there is no “structural flaw” which could allow, absent knowledge of plain-text related modifications – say “no, this key or key prefix could not produce this output for no input unmodified nonce”. If, for example, the plaintext was just a truly random bitstream of 16 MB with no particular quality beyond that, then, indeed, the modified nonce produced is fully just a truly random sampling of size 2GB, information theoretically indistinguishable from the output of a direct sampling without SKREM modifications.

So how can we deny the attacker the benefit of the known plain-text? And, in maniac mode, potentially even of “easily” knowing the GMT based used for encryption?

The idea is like this.. what if we used two (or three) SKREMA Alpha encrypted instances (so we get space for 2x or 3x times 16 MB per block), one two encrypt some parametrized mathematical statement and the other to encrypt some trapdoor which is used to decide if the statement is true or false?

Initially the author thought along the following lines. There exist mathematical statements which were/are very hard to verify and cannot be simply “brute-forced”. For example, Goldbach’s prime conjecture (GOLDBACH) which is still unresolved. Its statement can be expressed however in terms of a Verifier Turing Program (VERIFIER), named VGOLDBACH which, if it terminates, invalidates the conjecture and if it doesn’t, it confirms it (simply verifying all primes for the property incrementally up to infinity). Up to present day, it is unknown if VGOLDBACH terminates or not. And there are others: Is PRIMES in P? (Yes) Does  $P=NP$ ? (unknown) Do arbitrary strings  $x$  and  $y$  converge under 2-THUE rewrite rules? (unknown) Do all HYDRA battles terminate? (Yes) Is the parity bit of the number of ones in the binary representation of BUSYBEAVER(2,9) true? (No). Does a particular Turing machine HALT? (potentially undecidable).

Now, what if we were able to somehow parametrize a class of such statements - generally undecidable or at least very hard to decide -, in such a way, that, with the trapdoor

we encrypt using SKREMAAlpha, and they are easy to solve? We could then encrypt both the statement (or its parameter index) and the trapdoor with SKREMAAlpha (potentially with independent keys), and thus be able to produce a stream of pairs <INSTANCE, TRAPDOOR> which resolves to a bitstream. Since we choose both INSTANCE and TRAPDOOR we can make this bitstream arbitrary.

The general approach here is as follows:

1. We take some indexed class of hard mathematical statements or conjectures, known or suspected to be hard (preferably undecidable) in the general case.
2. We generate an instance of that class problem, in such a way that, given some Trapdoor bitstream we possess (or dynamically generate also), we are able to reason about some aspects of it fast, in order to get a 0/1 bit of insight.
3. We encrypt the sequence of, say fixed-sized Trapdoor bitstreams with key1 using SKREMAAlpha.
4. We also encrypt the sequence of also say fixed-sized instances of the hard problem with key2 also using SKREMAAlpha.
5. In non-maniac mode, we then simply choose the induced bitstream to represent a XOR mask to be used on the plain text.
6. For maniac-level security, one can encrypt not just the plain-text bits encoded in this fashion, but also the random nonce GMT. As per HUMINT claims, encrypting the GMT even modestly securely (with AES) makes even the original variant of SKREMA secure -- which does not mean such claims are to be taken as fact, but only as an indication that encryption before the KPRF step complicates the circuit for the attacker much more so than just plaintext encryption.
7. Even in maniac mode, for practicality, we recommend as follows: a 16MB bitstream (a pre-encryption key) is encrypted with the methods in this section. This key is then used to generate the randomness GMT as per the practical details in section 1F both at encryption and at decryption time. SKREMAAlpha in this context is then used just as a KPRF, producing an output which is a bitstream of arbitrary size. Then either (1) that bitstream is XORed with the modified GMT which encrypts the plain text or its representation as per points 1-5 if in hypermaniac mode; or (2) the bitstream is not published at all, but instead the output of SKREMAAlpha in decrypting "000..." over it is XORed with the plain text and published.

Note that we expect that points 5-7 above will give any automatic solver, including one learning correlations a much harder time than simply using SKREMAAlpha. This is claimed to be enough to make the approach secure until the turn of the current 21st century.

If our construction of <Hard Problem, Instance+Trapdoor generation> is sound, knowing key1 and key2 it is

trivial to run the VERIFIER program to retrieve the plain text bit. Also, we assume that the Hard Problem allows generation process to produce pairs which arbitrarily encode 1/0 and in either case the process is efficient (sufficiently fast). The VERIFIER is assumed efficient – polynomial.

However, for the attacker, the problem became potentially way more difficult: he now longer asks "Is there a secret key which when used on this output produces the plain text?". In fact, he has to ask: "Is there a pair of secret keys (key1, key2) which, when used in conjunction produce a sequence of (INSTANCE, TRAPDOOR) pairs which encode the plaintext?". Or, in maniac mode, "Do they encode a GMT which admits a secret key which together encode the plain text?". How is that different? It is now much harder and computationally consuming to hypothesize both "backwards", from the output modified nonce he sees, and, in maniac mode, also "forwards" to formulate and test correlations against an output he no longer easily sees.

How so? For the class of problem we will consider, in fact any INSTANCE has at least a TRAPDOOR producing 1 and at least a TRAPDOOR producing 0. So basically, he cannot gain anything by fixing via guessing an INSTANCE. In fact, for the class of problems we consider, even for a given trapdoor, the number of INSTANCES giving 1 is exponentially large, as is the larger number of INSTANCES giving 0. In fact, without either of these fixed, the collective pair <INSTANCE, TRAPDOOR> has a huge count even information theoretically wise. And for any particular guess, the attacker then needs to ask "can this pair be produced by SKREMAAlpha for some key?".

So, what can be a simple, clean choice of INSTANCE / TRAPDOOR? The obvious example is a RANDOM CIRCUIT built as follows, based on entropy created using SKREMAAlpha with some short key(s):

- Circuit consists of l layers.
- Each layer consists of some number of input gates q, and some number of output gates r. An EXPANDER has  $r = 2q$ , a CONTRACTOR has  $r = q/2$ . A MIXER has  $r=q$ .
- Each gate has constant fan in 4.
- Each gate (except the second MIXER stacked layer) is a dynamic balanced probabilistic random function (ie. like the one built in Step 4 using the strong bits).
- Layers are stacked two by two at a time like this:  
EXPANDER EXPANDER MIXER MIXER  
CONTRACTOR CONTRACTOR.
- Input indexes to gates are chosen randomly (wires), avoiding having too large fan-out for any single input: about 4-8.
- Inputs are random and represent TRAPDOOR, while circuit-construction entropy is also random and represents INSTANCE.

When used to hide the plain text, basically there can be 3 SKREMAAlpha, each for INSTANCE, TRAPDOOR and <plain-text XOR circuit output>.

When used to hide the GMT, either 1 or 2 SKREMAAlpha usages suffice: one for TRAPDOOR and optionally one for INSTANCE. The actual GMT used for encryption will be XORed with the output of this circuit.

Is anything better possible?

#### B. *Of maths, hydras, rewrites and busy beavers*

The desire is to build some sort of problem, which an automated systems cannot solve – or at least has a very hard time solving, at the very minimum considering current approaches.

Naturally, problems undecidable in the general case come to mind. But like NP-complete problems, there is no telling that being undecidable in the general case guarantees for a particular instance of that class of problems that it were hard. Solving Busy Beaver is an example for HALTING.

So, we moved on deeper to examine stuff AI/ML models are bad at, terribly bad at – and no, it's not creativity; its common sense and critical judgement.

For example, for an automated system, as far as currently implemented, as simple statement like “INDUCTION OVER N IS TRUE” or “STRUCTURAL INDUCTION IS TRUE” is something which must be taken as an axiom. It does not, in itself recognize such statement as factually true, without the need to limit reasoning about a theory containing it that at most that “it appears internally consistent; it does not self-contradict nor break other axioms”. So, we started looking for a problem where the answer involved something like “INVENT A TRUE AXIOM”. Unfortunately, the quest for such axioms – especially interesting ones, is difficult for human researchers as well. Consider just the AXIOM OF CHOICE, or NON-EUCLIDIAN GEOMETRIES. And, in fact, we don't just want a statement produced which by some human ingenuity is determined to be true/false (eg. IS PRIMES IN P? – true). We needed an indexable class of such statements AND a fixed or quickly buildable VERIFIER algorithm (Turing Machine) which, when presented with the INDEX and a TRAPDOOR, decided the statement. Also, we wanted both encryption and decryption to happen without any human in the loop.

Here we have to hand it to ChatGPT v4.5 for being able to pseudo-creatively extend the discussion about such issues in the context of cryptography, integrating ideas from the public domain of various domains of Computer Science marvelously. While it cannot be said to be strictly creative, the manner in which it chose to integrate HYDRA BATTLES as a response to our quest text on the above topics is amazing. If it were a human, we would call it ingenious. Anyhow, it's a show that simply mimicking how a very vastly knowledgeable panel of humans would talk about some topic in a particular context can lead to ideas which individual actual humans themselves find novel. So, kudos to LLM and ChatGPT v4.5 for that. Before using ChatGPT, the author himself did not know what a HYDRA BATTLE was.

#### C. *Hydra-game based encryption*

A HYDRA is a rooted, labeled, finite tree, coupled with a set of rewrite rules  $S$ . There are  $n$  nodes and  $m < n$  labels initially.

The set of rewrite rules is a set of functions  $S = \{f(x, \text{time}) \rightarrow \text{rooted, labeled, finite tree}\}$ . Here  $x$  is a HYDRA node, and time is a natural number. Note that  $f$  can consider the label of  $x$ , the structure of the tree around  $x$  (parent, grandparent) and has full access to the HYDRA tree. It is generally understood however to be simple and linear in input/output.

A HYDRA BATTLE is the action of choosing a specific **leaf** node  $x$  in the HYDRA Tree and a specific rewrite rule  $f$  and replacing the node  $x$  with  $f(\text{label}[x], \text{time}++)$ ,  $\text{label}[\text{parent}[x]]$ ). Note that  $f()$  can produce the empty set.

A HYRDA GAME is a sequence of HYDRA BATTLES, with time increasing by 1 between them.

A Kirby–Paris hydra [33] is a famous type of Hydra where *every game terminates* BUT *this fact is NOT PROVABLE in Peano Arithmetic (PA)*. Such a hydra is as follows:

- Labels are ignored (no labels).
- A single rewrite rule: When  $x$  is cut, remove it from its' parent subtree. Unless the parent of  $x$  is directly the root, also replace this *parent* subtree with  $t$  copies of itself (after having removed  $x$ ) in its grandparent, where  $t$  is the current time.
- If just the root remains – the game terminates and Hercules (the cutter) is victorious.

There are more complicated classes of hydras whose instances are known to ALL terminate, but the fact of that being so, to be unprovable even in ZFC (the mainstream Mathematical Theory of this century), like Buchholz hydra [34], Toshiyasu Arai hydra [35] and Ordinal-indexed hydras with collapsing functions [36].

The idea is that for a particular ordinal assignment to each possible rewrite context of the HYDRA, even with ordinals below epsilon zero (so of the form  $\text{Sum}(c[i] * \omega^{k[i]})$ ), it is easy to construct a satisfying Hydra and the rewrite rules so that the specified assignment Proves the Hydra is terminating. The converse however – given just a HYDRA to find an ordinal assignment to prove it terminates reduces from a known hard problem and is information-theoretically super polynomial in general. Also, for any HYDRA a random ordinal assignment is very improbable to prove it correct, and, also, it is that while few %-wise, there are many assignments which do prove it terminates.

For the HYDRA-based cryptography itself, construction and verification are easy: choose an arbitrary ordinal assignment for all possible contexts and then build the HYDRA and the rules rejecting rules which violate it.

The issue arises that the restriction that the VERIFIER terminates in polynomial time reduces the information theoretical complexity of the problem used as such – down to at most the satisfiability of a Boolean circuit, so NP-Complete. And like all NP-Complete based cryptography, this class of problem instances are by far not guaranteed to be immune from heuristic plus AI/ML SAT Solvers.

Nevertheless, in the context of its use to hide a GMT / plain-text in a SKREMA $\alpha$  scheme, its added complexity might significantly complicate Solvers – as opposed to merely solving a SKREMA $\alpha$  instance first, validating the solve, then solving a HYDRA instance or the other way around.

#### D. Rewrite Based Cryptography.

Apparently, the approach in subsection B is called Logic-Based Cryptography namely Rewrite Based Cryptography and Ordinal Complexity.

The quick failure of the Wagner-Magyarik Cryptosystem (1985) [37] in the general case points to the idea that simply picking rewrite rules at random is not sufficient to ensure security. More specifics are needed if Rewrite Based Cryptography is to add security value. Note that this is despite the fact the search problem of determining if a set of rewrite rules terminate on a given input is, of course, Undecidable in general (its equivalent trivially to HALTING).

Ultimately note that CRYPTOGRAPHY itself, at least in the current commonly understood model, is at most NP-Complete since the DECRYPTOR is considered fixed and must complete in polynomial time. This induces a Turing Machine of some maximal polynomial running time  $t$ , which can be translated to a Boolean circuit of size  $t^2$ , which can be translated to a Boolean formula of size  $O(t^2)$  – inverting which is NP-Complete with known plain text.

So, the battle is to generate strong, complicated circuits that AI/ML based solvers cannot handle. This relates to producing ciphers which induce such circuits, even if the solvers, like CyberMallory do not convert the programs to circuit in order to attack them.

One might be tempted to use a simple approach and just pick a random key, and then sample uniformly randomly a 3CNFSAT formula is satisfied by it, from all such which lie at the threshold or above it at circa 3.7-5 terms / literal. Unfortunately, this scheme in particular is not secure in practice. AI/ML solvers can pick up structure induced by the forced solution and use it to navigate to it much quicker than exponentially.

So, it seems, for truly provable unbreakable security, further advances in  $P = NP$  theory, or a fundamental change of the encryption / decryption model (eg. putting a human or a cat in the loop) are needed.

Finally, for practical usage today, and prospectively until the end of the 21st century, we expect that Step 7 of the

approach in Section A, with the 16MB key to it encrypted also with SKREMA $\alpha$  (not some more sophisticated rewrite scheme), together with a published XOR masked applied to the cipher text before actual encryption, we expect that this maniac-mode approach should be good enough against all adversaries, foreign and domestic, include state actors, including given prospective growth in their compute power, Moore's law (eg. photonic computing) and generally evolution of the parameters presented in Section 6 over the remainder of this century.

We conclude this paper here.

## IX. VITAE

Mircea Digulescu is an independent computer science researcher, software engineer, entrepreneur, military and intelligence enthusiast and amateur writer.

He has a PhD (ABD – All but Dissertation) from University of Bucharest, Department of Computer Science of Faculty of Mathematics and Computer Science, from which he also obtained both a Bachelors and a Master's degree in Computer Science. He gained recognition by being awarded medals and prizes at international contests and Olympiads in Computer Science, including 1 Bronze Medal at CEOI and 2 prizes at ACM SEERC. He was also a Div1 coder on Codeforces.com.

He has over 20 years of hands-on experience in the software industry, where he earned close to \$1M USD in revenue working as an Eastern Europe individual entrepreneur for various clients on non-research software engineering projects (which he has long spent entirely btw).

His research interests include Cryptography, Game Theory, Complexity Theory, Calculability Theory, Physical Limits of Computation and Algorithms and Data Structures.

Mircea Digulescu is also a libertarian, politically involved, social activist.

He is also a rather known military and intelligence enthusiast and somewhat popular strategist in the Back Sea Security area and beyond.

In this context, for those requiring or finding such traits useful, the motivation of Mircea Digulescu to put forth strong cryptography, in his capacity as an anti-authoritarian, pro-libertarian, persecuted and sometimes hunted computer scientist, leader and M&I strategist – here they are. To the question why make such powerful cryptography ideas public? Well.. Mircea believes the top-notch western block agencies already have or are developing SKREM-like ciphers (and probably had AI/ML enhanced variants of SKREMA $\alpha$ -like ciphers since before 2021). So he gives them nothing new. Cryptography, both in military and civilian application is largely a defensive tool and a privacy enabler. Given the current configuration of laws, applications thereof, concrete military and civil situation across the entire globe and nearby, the enormous quantity of idiots, ill-intended, evil, primitive or other such persons with various degrees of formal or de-facto power everywhere on this planet, of other idiots and dishonest people generally, Mircea

considers that giving powerful encryption to everyone is the most (and ultimately only) reliable way to make sure “good actors” and parties (as so understood by Mircea) have this dual-use tool available to them. “Bad” actors either already have it, or, if useful to other bad actors get it in form of hardware machines. So, according to Mircea’s assessment, publishing the ideas here gives potential foes no significant advantage (expect a little bit perhaps with ideas of sections 8 and 9 – but still not far away from arm’s reach for a savvy computer scientist) while giving the public at large, the cryptocurrency and Web3.0 community, and friendly actors of all walks of life, potentially a great advantage – leveling the tech field in cryptography (except in terms of compute power), all the way up to the top-beyond-leading-edge institutional state actors out there.

For those interested in Mircea’s hobby as a politically involved strategist and libertarian activist, he maintains a document called “Mircea Digulescu – Political Biography” [38] on his website [x] and has also authored two strategy advisory reports in 2021 [39] and 2024 [40] which he disseminated to several world leaders and states and which are rather popular with some people in the Military & Intelligence field as well as with some part of the public.

The most important peer-reviewed and prospective research papers (mostly in computer science) by Mircea Digulescu are also available as a list with abstract and links to full text on his website’s Research section [x].

Finally, visiting the authors personal website homepage [www.matsoft.ro](http://www.matsoft.ro) and browsing through the top bar menu items, brings up comprehensive details and documents pertaining to the authors various endeavors. Most documents are in English, some with Russian versions also available. Some portions and documents are in Romanian only. For fun reading, make sure to visit “Explanation of Love” and “The Little Big Game of Life” subsection on the “Beletristics” tab.

To contact the author, feel free to use the then-current contact data available on the Contact section of his personal website [42], or write to him at [mircea.digulescu@mail.ru](mailto:mircea.digulescu@mail.ru), [mircea.digulescu@gmail.com](mailto:mircea.digulescu@gmail.com) or via WhatsApp/Telegram at +40736.617.391. Except the email addresses which are dawning throughout time, other contact details may change.

## X. ACKNOWLEDGMENTS

No organizations funded the research presented in this paper. The author is currently an unaffiliated, independent researcher. The author's prior affiliation was University of Bucharest, Department of Computer Science of Faculty of Mathematics and Computer Science. Statement of interest: none.

LLMs, especially ChatGPT v4o and v4.5 played an important role in quick synthesis of literature and knowledge in various directions. ChatGPT v4o was used for quite a few simulations regarding elements of SKREMAAlpha including some aspects of the Random-G Turing Machine. Literary aspects have largely been confirmed via Google Search before being taken as fact, or not used directly anyhow.

Warm thanks for their existence and understanding to the few beautiful persons who inspired the author to strive to make the world a better place for them and whom he regards

as being synonymous with the meaning of the definition of love, to mention just a few in this context in random order: Nicolet’ q\_sinonim-obiectiv (and her sister Lanet’) from Debreceen 2022, q\_spatiotemporal from Polovragi 2022, Karina 2025, Anca Pana Balietti 1/2, Ctlna Ghiorgi q’ 2014, Tatyana Nevidina, Eliza and Eva Nan 2024, Anca Yerevan 2022, syno -2 and -2’ Yerevan 2022, Simona Nastase Calita 2005, syno kk02 from Moscow 2022, q\_sinonim-inghetata-fresh1 (de ciuperci) from Moscow 2022, q\_sinonim-park-geamantan from Moscow 2022, q\_sinonim-limonada-fresh2 (main) from Bucharest 2022, synonymous from LiWu1 from Bucharest 2022, and LiWu2 also Bucharest 2022, sofia rd.#0 from Bucharest 2022, syno rd.#26 from Nizhny Novgorod 2020, white daughter of David Yerevan 2022, Syuzi Yerevan 2021, Dee Demirbag (my romulanka), Tojet Tereza Mojet 2015 Moscow, UMS 2015 Moscow (remote), q\_secreta 2013 (remote) si 2022 Moscow, je\_veu2 (remote), Alina Pui de Urs 2014, qblu Istanbul 2022, Alexandra de pe dealul negru 2013, Svetlana 2510 from Nizhny Novgorod 2020, Alinochka8 and Valeria sombody2 from Nizhny Novgorod 2020, Belinda Carlisle (Severnaya Ossetia Allanina), Malena ml1 from Nizhny Novgorod 2021, o\_mic, Vika39red (remote), synonymous white from Bucharest bus Sun Stop 2025, synonymous aceea 2025 and cealalta 2025, powerful fresh synonymous Anna from strawberry 2025, syno -25 and syno -26 from Manheim 2022 (remotely), Anna from Latte 2026, Laura Pana, Elena Pana, Venezuela6, Bianca Pui1 Bucuresti din 2019, and last but not least Diana Paiu Ritzus and, with all the love possible, the beautiful latest addition, Alexandra Elena Petre aka ali rd.#-1111. This paper would never have existed without them.

A rough preprint of the contents of this paper has been published by the author in March 2026 [43].

## XI. DISCLAIMER

The content of this paper and the whatever linked software have been produced and are distributed in accordance with the “open-source” exemption for cryptographic items under the terms of the Wissegrad treaty [44] and are therefore exempt from arms and dual-use items licensing requirements in signatory states.

For avoidance of doubt, the mention of the 2022 event in this paper in this form is purely for argumentative scientific purposes. It is both legitimate – as the author did not agree to any NDA neither verbally nor in writing regarding - and also shows sufficient military professional courtesy to the respective counterparty – a grace that the author himself in large part did not enjoy shortly after that encounter, when he got Assanged or 1 year in Germany.

Finally, the author apologizes for the rather lengthy size of this paper – he did not have the time and resources to make it short. He also found the ideas herein rather timely so as to warrant publishing in current form.

## REFERENCES

- [1] Digulescu, M., 2021. Hiding Data in Plain Sight: Towards Provably Unbreakable Encryption with Short Secret Key and One-Way Functions. 10th International Conference on Information Technology Convergence and Services (ITCSE 2021), June 26~27, 2021, Sydney, Australia ISBN : 978-1-925953-43-5; Cryptology ePrint Archive. DOI: [10.13140/RG.2.2.34319.94887](https://ia.cr/2021/495). <https://ia.cr/2021/495>.
- [2] Digulescu, M., 2021. Applications of SKREM-like symmetric key ciphers. 10th International Conference on Information Technology Convergence and Services (ITCSE 2021), June 26~27, 2021, Sydney, Australia ISBN : 978-1-925953-43-5; Cryptology ePrint Archive. <https://ia.cr/2021/496>.
- [3] Daemen, J. and Rijmen, V., 1998, September. The block cipher Rijndael. In International Conference on Smart Card Research and Advanced Applications (pp. 277-284). Springer, Berlin, Heidelberg.
- [4] Zimand, M., 1986. On the topological size of sets of random strings. *Mathematical Logic Quarterly*, 32(6), pp.81-88.
- [5] Zimand, M., 2009. Extracting the Kolmogorov complexity of strings and sequences from sources with limited independence. arXiv preprint arXiv:0902.2141.
- [6] Levin, L.A., 2003. The tale of one-way functions. *Problems of Information Transmission*, 39(1), pp.92-103.
- [7] Armour, P.G., 2015. The chaos machine. *Communications of the ACM*, 59(1), pp.36-38.
- [8] McEliece, R.J. (1978). A Public-Key Cryptosystem Based on Algebraic Coding Theory" DSN Progress Report, 42-44, pp.114–116.
- [9] Bertoni, G., Daemen, J., Peeters, M., & Van Assche, G. (2013). Keccak. In *Advances in Cryptology—EUROCRYPT 2013* (pp. 313-316). Springer Berlin Heidelberg.
- [10] Barlik et al (2024). Differential cryptanalysis with SAT, SMT, MILP, and CP (IACR Cryptology ePrint Archive).
- [11] Harish, Rajesh Kumar, Indivar Gupta, R.K. Sharma. SMT Solver-Based Cryptanalysis of Block Ciphers (Springer/Applied Sciences).
- [12] So, J. (2023) Deep-Learning-Based Cryptanalysis of Lightweight Block Ciphers, *Entropy*, 25(7), 986. Available at: MDPI Entropy.
- [13] Naserelden, S., Alias, N., Altigani, A., Mohamed, A. and Badreddine, S. (2025). Advanced attacks on AES: A comprehensive review of side-channel, fault injection, machine learning and AI-based attacks, and quantum computing threats, *Journal of Engineering and Applied Sciences*, 9(4), pp. 2471–2486.
- [14] Yuan, Y., Wang, Y., Li, S. and Zhang, W. (2023) Improved Neural Differential Distinguisher Model for Speck32/64, *Applied Sciences*, 13(12), 6994. Available at: MDPI Applied Sciences.
- [15] Gohr, A. (2019). Improving Attacks on Round-Reduced Speck32/64 Using Deep Learning, *Advances in Cryptology – CRYPTO 2019*, pp. 150–179. Available at: IACR Cryptology ePrint Archive.
- [16] Biryukov, A., Dunkelman, O., Keller, N., Khovratovich, D. and Shamir, A., 2010, May. Key recovery attacks of practical complexity on AES-256 variants with up to 10 rounds. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques* (pp. 299-319). Springer, Berlin, Heidelberg.
- [17] RANDRIAMBOLOLONA Hugues, ANSSI & Telecom Paris (2025). CryptoDB.EUROCRYPT2025.Doi: 10.1007/978-3-031-91095-1\_12.
- [18] Zhao, Y. and Sun, H. (2021) 'Expand-and-Randomize: An Algebraic Approach to Secure Computation', *Entropy*, 23(11), 1461.
- [19] Oliveira I.C.,Santhanam R.,Tell R.(2022), Expander-Based Cryptography Meets Natural Proofs,Computational Complexity 31(1).
- [20] Anand, E. (2025). Towards the Pseudorandomness of Expander Random Walks for Read-Once ACC0 circuits, arXiv preprint arXiv:2501.07752. Available at: arXiv.org.
- [21] Ryabko B., and Zhuravlev V., 2020. Pseudo-random number generators with proven statistical properties. The 9th Workshop on current trends in Cryptology, CTCrypt 2020.
- [22] Digulescu M. (2024), Pitchdeck for S-Company, Personal Website. Available at [https://www.matsoft.ro/Matsoft\\_Startups62\\_S-Company\\_113.html](https://www.matsoft.ro/Matsoft_Startups62_S-Company_113.html).
- [23] Digulescu M. (2024), Pitchdeck for M-Company, Personal Website. Available at [https://www.matsoft.ro/Matsoft\\_Startups62\\_M-Company\\_113.html](https://www.matsoft.ro/Matsoft_Startups62_M-Company_113.html).
- [24] Digulescu M., Personal GitHub profile. Available at <https://github.com/mircead85>.
- [25] Tevador (2023). RandomX: Proof of work algorithm based on random code execution. GitHub. Available at: GitHub - tevador/RandomX.
- [26] Digulescu, M. Codeforces.com profile. Available at: <https://www.codeforces.com/profile/mircea85>.
- [27] Knuth, D.E. (2011) *The Art of Computer Programming, Volumes 1-4A Boxed Set*. 3rd edn. Reading, MA: Addison-Wesley. Available at: Pearson Education.
- [28] Rogaway, P. (2011) 'Evaluation of Some Blockciphers-Modes of Operation', Citeseer. Available at: Web.cs.ucdavis.edu.
- [29] Katz, J. and Lindell, Y. (2020) *Introduction to Modern Cryptography*. 3rd edn. Boca Raton: CRC Press. Available at: Lindell's Course Resources.
- [30] Digulescu, M.A. (2019). Towards Solving NP-Complete and Other Hard Problems Efficiently in Practice, ResearchGate. Available at: ResearchGate. Available at: [https://www.matsoft.ro/Matsoft\\_Research62\\_Computer\\_Science\\_113.html](https://www.matsoft.ro/Matsoft_Research62_Computer_Science_113.html).
- [31] Digulescu, M.A. (2025). Towards Solving NP-Complete and other Hard Problems Efficiently in Practice, GRS Publisher, pp. 14–37. Available at: GRS Publisher.
- [32] Digulescu, M.A. (2021). Interesting systemic effects of screen blink features of Opera Browser on Windows. Personal YouTube channel. Available at <https://www.youtube.com/watch?v=zRsQycagoT0>.
- [33] Kirby, L. and Paris, J. (1982) 'Accessible independence results for Peano Arithmetic', *Bulletin of the London Mathematical Society*, 14(4), pp. 285–293. Available at: Oxford Academic.
- [34] Buchholz, W. (1987) An independence result for (P11-CA)+BI, *Annals of Pure and Applied Logic*, 33(2), pp. 131–155. Available at: ScienceDirect.
- [35] Arai, T. (2018) Hydra games for recursively Mahlo operations, arXiv preprint arXiv:1808.04522. Available at: arXiv.org.
- [36] Rathjen, M. (2006) 'The Art of Ordinal Analysis', *International Congress of Mathematicians*, Vol. II, pp. 45–69.
- [37] Levy-dit-Vehel, F. and Guerin, R. (1993) 'A Note on the Security of the Wagner-Magyarik Public-Key Cryptosystem', *Advances in Cryptology – EUROCRYPT '93*, pp. 431–435.
- [38] Digulescu, M.A. (2024). Political Biography. Personal Website. Available at <https://www.matsoft.ro>.
- [39] Digulescu, M.A. (2021). 21st Century Strategy. Personal Website. Available at [https://www.matsoft.ro/Matsoft\\_BetterWorldProject62\\_Achieving\\_Liberty\\_113.html](https://www.matsoft.ro/Matsoft_BetterWorldProject62_Achieving_Liberty_113.html).
- [40] Digulescu, M.A. (2024). Strategy Report 2024 Update. Personal Website. Available at [https://www.matsoft.ro/Matsoft\\_BetterWorldProject62\\_Achieving\\_Liberty\\_113.html](https://www.matsoft.ro/Matsoft_BetterWorldProject62_Achieving_Liberty_113.html).
- [41] Digulescu, M.A. Personal Website Research in Computer Science section. Available at: [https://www.matsoft.ro/Matsoft\\_Research62\\_Computer\\_Science\\_113.html](https://www.matsoft.ro/Matsoft_Research62_Computer_Science_113.html).
- [42] Digulescu, M.A. (2026). Personal Website Contact Section. Available at: [https://www.matsoft.ro/Matsoft\\_Contact62.html](https://www.matsoft.ro/Matsoft_Contact62.html).
- [43] Digulescu, M.A. (2026). SKREM Alpha – A secure and practical SKREM-like symmetric key cipher, its premises and usages. Academia.edu. Available at Academia.edu and [https://www.matsoft.ro/Matsoft\\_Research62\\_Computer\\_Science\\_113.html](https://www.matsoft.ro/Matsoft_Research62_Computer_Science_113.html).
- [44] Wassenaar Arrangement General Software Note, in List of Dual-Use Goods and Technologies and Munitions List. Vienna: Secretariat of the Wassenaar Arrangement, p.4. Available at: Wassenaar.org.

